



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**Soul Food Ai: Sistema de
Gestión Nutricional con
Integración de Modelos de
Lenguaje de Gran Escala para
la Generación y Evaluación de
Dietas Personalizadas**



Presentado por Pablo Velasco Martín
en Universidad de Burgos — 11 de junio
de 2026

Tutor: Sandra Rodríguez Arribas



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



Sandra Rodriguez Arribas, profesora del departamento de Ingeniería Informática, área de Lenguajes y Sistemas Informáticos.

Expone:

Que el alumno D. Pablo Velasco Martín, con DNI 71955711G, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado Sistema de Gestión Nutricional basado en ASP.NET Core con Integración de Modelos de Lenguaje de Gran Escala (LLM) para la Generación y Evaluación de Dietas Personalizadas.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 11 de junio de 2026

Vº. Bº. del Tutor:

D. nombre tutor

Resumen

Este Trabajo de Fin de Grado presenta SoulFoodAI, una aplicación web integral diseñada para resolver la dificultad de mantener una alimentación saludable y variada mediante la personalización inteligente. Su objetivo principal es ofrecer una solución empática frente a los sistemas tradicionales, adaptándose al estado físico y emocional del usuario mientras aprovecha su despensa real para evitar el desperdicio alimentario.

Una característica diferenciadora de SoulFoodAI es la integración nativa del motor de inteligencia artificial Google Gemini 2.5 Flash y la base de datos colaborativa Open Food Facts. Esta sinergia permite la generación automatizada de recetas dinámicas y la catalogación precisa de alimentos, evaluando variables de estilo de vida para diseñar planes nutricionales altamente adaptables.

El desarrollo se ha abordado con un fuerte énfasis en la calidad del software, la seguridad y la escalabilidad. Se ha implementado una arquitectura moderna basada en Angular para el *frontend* y ASP.NET Core para el *backend* (API REST), utilizando bases de datos en Azure y contenedores Docker para garantizar un despliegue portable y reproducible. La metodología de trabajo ha seguido prácticas ágiles (Scrum), respaldada por un riguroso control de calidad (SonarCloud, Vitest, xUnit) y control de versiones mediante Git y GitHub.

Descriptores

Planificación nutricional, Inteligencia Artificial, Google Gemini, Open Food Facts, aplicación web, Angular, ASP.NET Core, API REST, Docker, calidad del software, desarrollo ágil.

Abstract

This Bachelor's Thesis presents SoulFoodAI, a comprehensive web application designed to solve the difficulty of maintaining a healthy and varied diet through intelligent personalization. Its main objective is to offer an empathetic solution compared to traditional applications, adapting to the user's physical and emotional state while leveraging their actual pantry to prevent food waste.

A differentiating feature of SoulFoodAI is the native integration of the Google Gemini 2.5 Flash artificial intelligence engine and the Open Food Facts collaborative database. This synergy enables the automated generation of dynamic recipes and accurate food cataloging, evaluating lifestyle variables to design highly adaptable nutritional plans.

The development has been approached with a strong emphasis on software quality, security, and scalability. A modern architecture has been implemented, based on Angular for the frontend and ASP.NET Core for the backend (REST API), utilizing Azure databases and Docker containers to ensure a portable and reproducible deployment. The work methodology has followed agile practices (Scrum), supported by rigorous quality control (SonarCloud, Vitest, xUnit) and version control via Git and GitHub.

Keywords

Nutritional planning, Artificial Intelligence, Google Gemini, Open Food Facts, web application, Angular, ASP.NET Core, REST API, Docker, software quality, agile development.

Índice general

Índice general	iii
Índice de figuras	v
Índice de tablas	vii
1 Introducción	1
1.1. Problema detectado y motivación	1
1.2. Materiales entregados	2
2 Objetivos del proyecto	5
2.1. Objetivos funcionales	5
2.2. Objetivos técnicos	6
3 Conceptos teóricos	9
3.1. Arquitectura cliente-servidor	9
3.2. La capa de presentación: aplicaciones de página única y diseño <i>responsive</i>	10
3.3. La capa servidor: patrón MVC e inyección de dependencias .	11
3.4. Comunicación entre cliente y servidor: HTTP, REST y JSON	12
3.5. Autenticación, Autorización y Criptografía	13
3.6. Persistencia de datos: ORM y migraciones	16
3.7. Integración de servicios mediante APIs externas	16
3.8. Conceptos de nutrición y dietética	17
3.9. Objetivos nutricionales y patrones dietéticos	19
4 Técnicas y herramientas	21

4.1. Tecnologías de <i>Backend</i> y Persistencia	21
4.2. Tecnologías de <i>Frontend</i> e Interfaz	22
4.3. Inteligencia Artificial y Servicios Externos	23
4.4. Gestión del Proyecto, Pruebas y Calidad	24
4.5. Infraestructura y Despliegue	27
5 Aspectos relevantes del desarrollo del proyecto	29
5.1. Inicio del proyecto y decisiones de diseño	29
5.2. Evolución del modelo de datos	30
5.3. Integración con fuentes externas de información nutricional .	35
5.4. Integración del motor de inteligencia artificial	37
5.5. Desarrollo de la interfaz de usuario	44
6 Trabajos relacionados	47
6.1. MyFitnessPal	47
6.2. Eat This Much	47
6.3. MacroFactor	48
6.4. Asistentes nutricionales basados en inteligencia artificial . . .	48
6.5. Comparativa de funcionalidades	48
6.6. Posicionamiento y valor añadido de SoulFoodAI	49
7 Conclusiones y líneas de trabajo futuras	51
7.1. Conclusiones	51
7.2. Líneas de trabajo futuras y viabilidad comercial	52
Bibliografía	53

Índice de figuras

3.1. Esquema general de la arquitectura cliente-servidor. Elaboración propia.	10
3.2. Flujo de autenticación basado en JSON Web Token (JWT). Elaboración propia.	15
4.1. Tablero Kanban utilizado para la gestión y seguimiento iterativo del proyecto en Zube. Elaboración propia.	24
4.2. Interfaz Swagger utilizada para documentar y probar los <i>end-points</i> de la API REST. Elaboración propia.	25
4.3. Ejecución de las baterías de pruebas unitarias del <i>backend</i> mediante xUnit. Elaboración propia.	26
4.4. Panel de control de SonarCloud mostrando el estado de calidad del código, deuda técnica y cobertura de pruebas del proyecto. Elaboración propia.	27
4.5. Panel de recursos y servicios del <i>backend</i> alojados en Microsoft Azure. Elaboración propia.	28
5.1. Diagrama entidad-relación del modelo inicial de SoulFoodAI. . .	31
5.2. Diagrama entidad-relación del modelo final de SoulFoodAI. . .	34
5.3. Flujo de búsqueda, transformación e incorporación de ingredientes mediante Open Food Facts.	36
5.4. Arquitectura general del proceso de generación de recetas mediante Gemini 2.5 Flash.	38
5.5. Plantilla base del <i>prompt</i> utilizado en el módulo CreateRecipeAI. . .	39
5.6. Proceso de ajuste automático de cantidades mediante el <i>endpoint</i> AdjustDayMacros.	41
5.7. Proceso de análisis del reporte semanal y generación automática de la siguiente planificación nutricional.	43

5.8. Paso de métricas físicas del proceso de <i>onboarding</i>	44
5.9. Panel principal con el calendario interactivo y las tarjetas de recetas disponibles para arrastrar y soltar.	45

Índice de tablas

4.1. Comparativa técnica y económica de APIs de modelos de lenguaje.	23
6.1. Comparativa de funcionalidades entre aplicaciones del sector y SoulFoodAI.	49

Capítulo 1

Introducción

La alimentación es uno de los pilares fundamentales de la salud y el bienestar personal. Sin embargo, mantener una dieta equilibrada sigue siendo un reto cotidiano. Históricamente, el sector de la nutrición digital ha estado fuertemente enfocado en herramientas restrictivas, diseñadas exclusivamente para la pérdida de peso o para el conteo estricto de calorías, dejando de lado a una inmensa mayoría de la población que simplemente busca aprender a comer bien de forma sostenible.

El propio nombre del proyecto, *SoulFoodAI*, nace de una premisa fundamental que compone todo el imaginario de la aplicación: «*Alimentar el cuerpo con cuidado es también una manera de alimentar el ALMA*». El objetivo trasciende la mera nutrición matemática; se trata de que el usuario se sienta a gusto comiendo, forjando una relación de paz con la comida. No es una aplicación diseñada únicamente para hacer dietas de bajada de peso, sino para llevar un estilo de vida verdaderamente saludable, descubrir nuevas ideas de recetas y disfrutar del proceso, utilizando los ingredientes que al usuario le gustan y que compra habitualmente en el supermercado.

1.1. Problema detectado y motivación

La motivación principal de este proyecto surge de la experiencia propia frente a la dificultad de mantener una alimentación saludable y variada cuando uno se independiza por primera vez. En este contexto, se ha detectado que las aplicaciones tradicionales presentan una serie de problemas fundamentales que las hacen ineficaces a largo plazo:

- **Desconexión con la realidad de la despensa:** Suelen proporcionar recetas predefinidas que exigen comprar ingredientes aleatorios o muy específicos, los cuales se utilizan una sola vez y terminan pudriéndose al fondo de la nevera. No tienen en cuenta qué compra realmente el usuario en su día a día.
- **Enfoque restrictivo y matemático:** La inmensa mayoría de las herramientas comerciales están orientadas exclusivamente al conteo estricto de calorías y a dietas de bajada de peso, ignorando a quienes simplemente quieren aprender a comer de forma saludable sin presiones.
- **Falta de adaptabilidad emocional y física:** Imponen planes rígidos que no se adaptan al estado del usuario. Si una persona tiene una semana de alto estrés, ha dormido mal o se siente con poca energía, la aplicación tradicional no es capaz de comprenderlo ni de flexibilizar sus recomendaciones.

SoulFoodAI busca solucionar estos problemas ofreciendo una gran flexibilidad. El sistema está diseñado para cualquier perfil: ya sea un atleta de alto rendimiento que necesita cuadrar sus macronutrientes, una persona que busca adelgazar, o simplemente un joven recién independizado que no sabe cocinar y carece de un objetivo calórico estricto. La aplicación comprende al usuario, evalúa su estado cada semana y se adapta a él, ayudándole a comer de forma saludable y variada con los ingredientes de los que ya dispone, acompañándolo paso a paso en la creación de hábitos reales y duraderos.

1.2. Materiales entregados

Para complementar la lectura de esta memoria, se han adjuntado varios recursos que permiten revisar a fondo el resultado del Trabajo de Fin de Grado:

Documentación (Memoria y Anexos): Un documento separado donde se detalla la planificación del trabajo, el catálogo completo de requisitos, los diagramas de arquitectura, el manual de usuario y el análisis de sostenibilidad.

Código fuente: Todo el código que da vida a SoulFoodAI (*frontend* y *backend*) ha sido empaquetado y entregado. Si se desea consultar el historial de desarrollo, el repositorio está alojado en GitHub: <https://github.com/pabloVelascoMR/SoulFoodAI>

Despliegue y Vídeo de ejemplo: Para que el tribunal pueda interactuar con la herramienta sin compilar el código, se provee el enlace al despliegue en producción (<https://soul-food-ai-app.vercel.app>). Como apoyo, se incluye una carpeta con vídeos en el pendrive demostrando su funcionamiento real.

Calidad del Software: Los resultados de las pruebas automáticas, el análisis de posibles vulnerabilidades y la cobertura de código pueden auditar públicamente desde el panel del proyecto en SonarCloud: https://sonarcloud.io/dashboard?id=pabloVelascoMR_SoulFoodAI

Capítulo 2

Objetivos del proyecto

Este capítulo describe de forma precisa los objetivos que se persiguen con el desarrollo de SoulFoodAI. Se distingue entre los objetivos funcionales, que definen qué debe hacer la aplicación desde el punto de vista del usuario, y los objetivos técnicos, que establecen los requisitos de calidad, arquitectura y tecnología que debe cumplir la solución construida.

2.1. Objetivos funcionales

El objetivo general del proyecto consiste en desarrollar una plataforma web de planificación nutricional personalizada que acompañe al usuario a lo largo de todo su ciclo alimentario semanal, desde la configuración inicial de su perfil hasta la adaptación continua de su dieta en función de su evolución real.

Para alcanzar este objetivo general se definen los siguientes objetivos funcionales específicos:

- Implementar un proceso de *onboarding* que recoja los datos biométricos, el nivel de actividad física, el objetivo nutricional, el patrón dietético y las intolerancias alimentarias del usuario, calculando automáticamente su tasa metabólica basal y sus objetivos calóricos y de macronutrientes mediante la fórmula de Mifflin–St Jeor.
- Proporcionar un catálogo de ingredientes enriquecido mediante la integración con la API pública de Open Food Facts [34], permitiendo al usuario construir una despensa personalizada con productos reales de supermercado.

- Ofrecer un sistema de generación automática de recetas mediante inteligencia artificial, adaptadas estrictamente a los ingredientes disponibles en la despensa, el patrón dietético y las intolerancias registradas en el perfil del usuario.
- Implementar un calendario nutricional semanal interactivo que permita al usuario organizar sus comidas mediante *drag and drop* y visualizar en tiempo real el cumplimiento de sus objetivos de calorías y macronutrientes.
- Desarrollar un módulo de cuadre automático de macronutrientes diarios asistido por inteligencia artificial, capaz de ajustar las cantidades de los ingredientes de las recetas seleccionadas para aproximar el resultado real a los objetivos nutricionales definidos para cada jornada.
- Implementar un sistema de reporte semanal que recoja la experiencia subjetiva del usuario —nivel de hambre, energía, calidad del sueño y adherencia al plan— y utilice dicha información para generar, mediante inteligencia artificial, una nueva planificación nutricional adaptada a su evolución real.

2.2. Objetivos técnicos

Más allá de las funcionalidades visibles para el usuario, el proyecto persigue un conjunto de objetivos técnicos que definen los estándares de calidad, arquitectura y tecnología de la solución:

- Diseñar e implementar una arquitectura cliente-servidor desacoplada, con un *backend* basado en ASP.NET Core que exponga una API REST segura y sin estado, y un *frontend* desarrollado en Angular que consuma dicha API de forma asíncrona.
- Modelar una base de datos relacional en Microsoft SQL Server utilizando el enfoque *Code First*, garantizando la integridad referencial, la normalización del esquema y la trazabilidad de los datos históricos mediante borrado lógico.
- Integrar el modelo de lenguaje (LLM) Gemini 2.5 Flash de Google de forma controlada y segura, actuando el *backend* como intermediario encargado de construir el contexto de cada petición, validar las respuestas obtenidas y garantizar la coherencia de los datos nutricionales calculados.

- Aplicar técnicas de *prompt engineering* y salida estructurada en JSON para minimizar la tasa de errores en las respuestas del modelo y garantizar su procesamiento automático por parte de la aplicación.
- Implementar un sistema de autenticación seguro basado en JSON Web Tokens (JWT) [15] que garantice el control de acceso a todos los recursos protegidos de la API.
- Desplegar la aplicación en un entorno de producción real utilizando Microsoft Azure para el *backend* y la base de datos, y Vercel para el *frontend*, garantizando la disponibilidad pública de la plataforma, y apoyándose en la *dockerización* de los componentes para facilitar su ejecución.
- Garantizar la robustez y calidad del software mediante el diseño e implementación de pruebas unitarias y de integración (xUnit, Vitest), integrando además herramientas de análisis estático continuo como SonarCloud para el control de vulnerabilidades y la medición de la cobertura de código (*Code Coverage*).
- Aplicar una metodología de desarrollo ágil basada en Scrum [36], con iteraciones centradas en módulos funcionales concretos y revisiones periódicas del modelo de datos y la arquitectura.

Capítulo 3

Conceptos teóricos

El desarrollo de SoulFoodAI requiere integrar conocimientos de dos disciplinas bien diferenciadas. En el plano técnico, se aplican los fundamentos de la ingeniería de software moderna para construir una arquitectura web robusta y segura. En el plano funcional, el núcleo de la aplicación se rige por principios consolidados de nutrición y dietética. A continuación, se presentan los conceptos teóricos clave de ambas áreas, esenciales para comprender las decisiones y tecnologías implementadas en el resto del documento.

3.1. Arquitectura cliente-servidor

La arquitectura cliente-servidor es el modelo de distribución de responsabilidades más extendido en el desarrollo de aplicaciones web. En él participan dos roles bien diferenciados: el **cliente**, que inicia las peticiones y presenta los resultados al usuario final, y el **servidor**, que las atiende ejecutando la lógica de la aplicación, accediendo a los datos y devolviendo las respuestas correspondientes [42].

La separación entre ambos roles tiene como consecuencia directa que múltiples clientes (con distintos sistemas operativos, navegadores o tamaños de pantalla) puedan consumir los mismos recursos sin que el servidor necesite conocer sus particularidades. Para hacer posible esta independencia del cliente, toda la lógica de negocio queda centralizada exclusivamente en el servidor, lo que facilita el mantenimiento, refuerza la seguridad y permite escalar la infraestructura de forma independiente [39].

En el contexto de SoulFoodAI, la implementación de esta arquitectura (ilustrada en la Figura 3.1) se materializa dividiendo el sistema en dos grandes

bloques: un *frontend* interactivo que actúa como cliente, y un *backend* que orquesta los servicios, comunicándose entre sí mediante peticiones estándar HTTP.

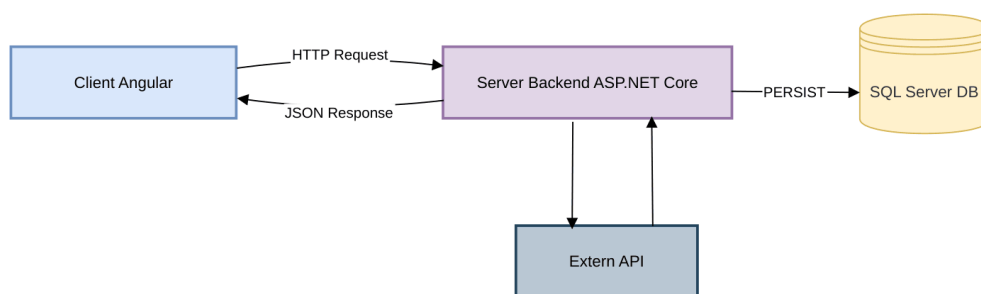


Figura 3.1: Esquema general de la arquitectura cliente-servidor. Elaboración propia.

Definido el modelo general, los dos apartados siguientes describen cómo se organiza internamente cada uno de los dos extremos: el cliente y el servidor.

3.2. La capa de presentación: aplicaciones de página única y diseño *responsive*

Una *Single Page Application* (SPA) es una aplicación web que carga un único documento HTML y actualiza su contenido dinámicamente sin recargar el navegador [31]. A diferencia de las aplicaciones tradicionales, el enrutamiento y la renderización son responsabilidad del cliente (JavaScript), ofreciendo una experiencia mucho más rápida y fluida.

El diseño *responsive* asegura que esta interfaz se adapte correctamente a cualquier tamaño de pantalla o dispositivo [19]. Para ello se emplean cuadrículas flexibles, imágenes adaptables y *media queries* (reglas CSS condicionales), eliminando la necesidad de desarrollar versiones separadas para móvil y escritorio.

Protección de rutas en cliente (*Guards*)

Dado que en una SPA la navegación entre vistas ocurre en el propio navegador, es necesario implementar mecanismos de control de acceso en el lado del cliente. Los guardianes de rutas (*Guards*) actúan como interceptores que evalúan una condición (típicamente, si existe un *token* de sesión válido) antes de permitir el acceso a una vista protegida. Si la evaluación falla, el guardián bloquea la navegación y redirige al usuario a la pantalla de inicio de sesión o muestra un error.

3.3. La capa servidor: patrón MVC e inyección de dependencias

Una vez establecido el modelo general cliente-servidor, conviene entender cómo se organiza internamente el servidor. Dos conceptos son fundamentales para ello: el patrón MVC, que define la estructura del código, y la inyección de dependencias, que gestiona cómo se relacionan sus componentes.

El patrón MVC

El patrón *Model-View-Controller* (MVC) separa la aplicación en tres capas lógicas [9]: el **Modelo** (datos y lógica de negocio), la **Vista** (representación visual de los datos, que en arquitecturas desacopladas recae en el cliente) y el **Controlador** (intermediario que recibe las peticiones, orquesta las llamadas al modelo y devuelve la respuesta correspondiente). Esta división facilita enormemente el mantenimiento y la escalabilidad del sistema.

El patrón Data Transfer Object (DTO)

Al comunicar el controlador con el cliente, es una mala práctica exponer directamente los objetos del modelo de base de datos (**entidades**). Para evitarlo, se utiliza el patrón *Data Transfer Object* (DTO). Un DTO es un objeto simple cuya única finalidad es transportar los datos estrictamente necesarios para una operación concreta a través de la red, ocultando información sensible (como campos internos de seguridad) y reduciendo el tamaño del envío.

Inyección de dependencias

La Inyección de Dependencias (*DI*) es un principio de diseño mediante el cual un objeto recibe las dependencias que necesita desde el exterior, en lugar de crearlas por sí mismo [20]. Un contenedor central se encarga de instanciar y suministrar estas dependencias en el constructor de cada clase.

Esto garantiza el desacoplamiento del código y facilita enormemente las pruebas unitarias, permitiendo inyectar objetos simulados (*mocks*) en lugar de implementaciones reales. Los servicios pueden registrarse con distintos ciclos de vida: *Transient* (nueva instancia cada vez), *Scoped* (una por cada petición HTTP) o *Singleton* (instancia única global).

3.4. Comunicación entre cliente y servidor: HTTP, REST y JSON

Con los dos extremos definidos, el siguiente paso es entender cómo se comunican. Esa comunicación descansa sobre tres pilares que se construyen uno sobre otro: el protocolo HTTP define las reglas del intercambio de mensajes, REST establece cómo se diseña la API sobre HTTP, y JSON es el formato en que se serializa la información.

El protocolo HTTP y el formato JSON

El *Hypertext Transfer Protocol* (HTTP) es la base de la comunicación web [6]. Funciona mediante peticiones donde el cliente indica un **método** (GET para consultar, POST para crear, PUT o PATCH para actualizar, y DELETE para borrar) sobre una **URI** concreta, y el servidor contesta con un **código de estado** numérico (2xx para éxito, 4xx para errores en el cliente, y 5xx para errores internos del servidor).

Para enviar y recibir la carga útil (*payload*) de estas peticiones se emplea el formato **JSON** (*JavaScript Object Notation*) [4]. Su adopción generalizada en lugar de XML se debe a su sintaxis más ligera, basada en pares clave-valor y orientada a objetos, lo que permite un procesamiento muy rápido y natural en el desarrollo web.

El estilo arquitectónico REST

Representational State Transfer (REST) define cómo diseñar la API sobre HTTP [7]. Sus principios dictan que la comunicación no debe guardar

estado de sesión (*stateless*), y que la manipulación de recursos debe hacerse utilizando exclusivamente los métodos HTTP estándar de forma uniforme. Un **endpoint** es la combinación de una de estas rutas y su método HTTP asociado.

Cross-Origin Resource Sharing (CORS)

Dado que la arquitectura consta de un *frontend* y un *backend* totalmente separados, estos suelen ejecutarse en dominios o puertos distintos (orígenes diferentes). Por motivos estrictos de seguridad, los navegadores web bloquean por defecto cualquier petición HTTP realizada a un origen distinto al de la página actual mediante una política conocida como el Mismo Origen (*Same-Origin Policy*).

Para permitir esta comunicación cruzada de forma legítima, el servidor debe implementar **CORS**. Esta política se basa en el envío de cabeceras HTTP especiales que autorizan explícitamente a uno o varios orígenes externos concretos a interactuar de manera segura con su API REST.

3.5. Autenticación, Autorización y Criptografía

Conviene diferenciar claramente dos conceptos fundamentales en la seguridad de la aplicación: la **autenticación** verifica la identidad del usuario para confirmar quién es, mientras que la **autorización** define qué recursos o acciones tiene permitidas una vez identificado.

Criptografía y *Hashing* de contraseñas

Como regla inquebrantable de ciberseguridad, las contraseñas nunca se almacenan en texto plano en la base de datos. Para protegerlas se aplican funciones de *hashing* criptográfico, unas funciones matemáticas que transforman la contraseña original en una cadena alfanumérica de longitud fija de forma estrictamente unidireccional (es matemáticamente imposible revertir el proceso).

Además, a cada contraseña se le añade un valor aleatorio único denominado *salt* antes de procesarla. Esto garantiza que dos contraseñas idénticas no generen nunca el mismo *hash*, neutralizando eficazmente los ataques basados en tablas precalculadas (*rainbow tables*). En el momento de iniciar sesión, el servidor no necesita descryptar la clave; simplemente aplica el

mismo proceso de *hashing* a la contraseña introducida y comprueba si el resultado coincide con el *hash* almacenado en la base de datos.

JSON Web Token (JWT)

Una vez que el usuario se ha autenticado con éxito, en arquitecturas REST sin estado (*stateless*) se utiliza el estándar *JSON Web Token* (JWT) [15] para mantener la sesión abierta. El servidor genera un token codificado en Base64url que consta de tres bloques: una cabecera, una carga útil (con las afirmaciones o *claims* del usuario, como su identificador y la caducidad) y una firma digital generada utilizando una clave privada y secreta del servidor.

El cliente recibe este token y, a partir de ese momento, lo adjunta en la cabecera de autorización de cada petición posterior que realice a la API. El servidor verifica matemáticamente la validez de la firma digital del token para autorizar la petición, eliminando así la necesidad de consultar la base de datos para recuperar la sesión del usuario en cada llamada [16].

La Figura 3.2 ilustra gráficamente este flujo.

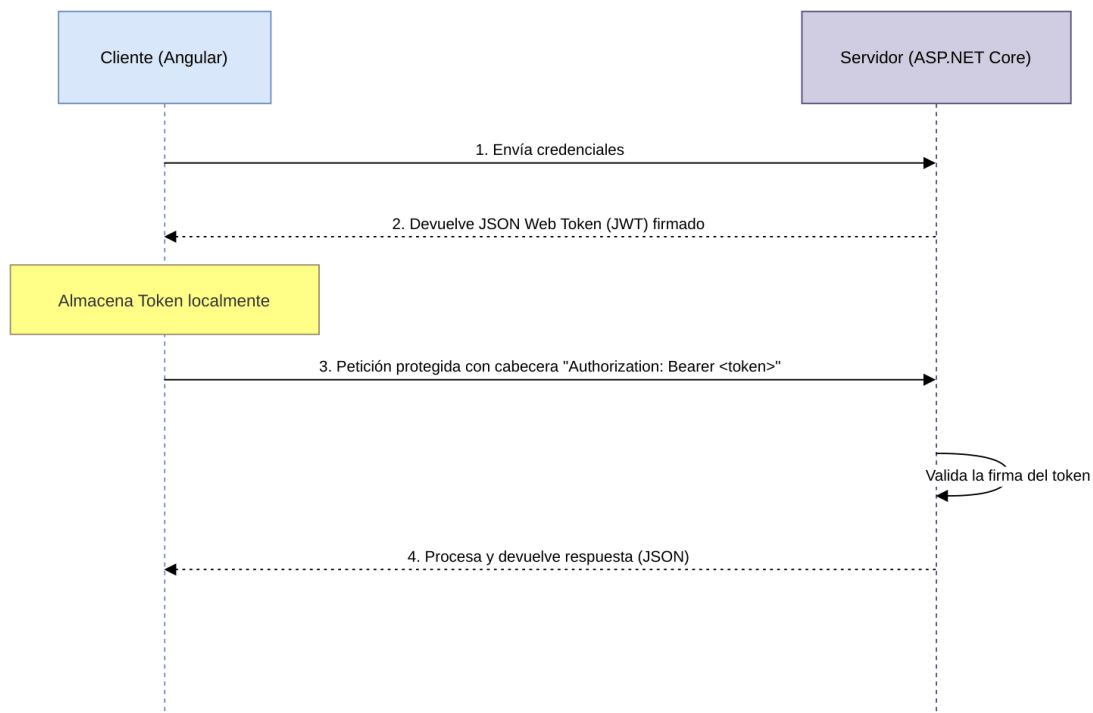


Figura 3.2: Flujo de autenticación basado en JSON Web Token (JWT).
Elaboración propia.

3.6. Persistencia de datos: ORM y migraciones

Un *Object-Relational Mapper* (ORM) es una herramienta que establece una correspondencia automática entre las clases de un lenguaje orientado a objetos y las tablas de una base de datos relacional [18]. Su objetivo es permitir al desarrollador operar sobre los datos mediante objetos del dominio sin necesidad de escribir sentencias SQL manualmente; el ORM se encarga de traducir esas operaciones a las instrucciones de base de datos equivalentes.

En el enfoque **Code First**, el proceso se invierte respecto al desarrollo tradicional: el desarrollador define primero las clases del dominio (denominadas **entidades**) y es el ORM quien genera el esquema de la base de datos a partir de ellas.

Los cambios en el modelo de datos a lo largo del tiempo se gestionan mediante **migraciones**. Una migración es un archivo de código generado automáticamente que describe la diferencia entre el estado actual del esquema y el nuevo estado deseado. Aplicar una migración actualiza la base de datos de forma incremental, y los archivos pueden versionarse junto con el resto del código fuente, lo que facilita la trazabilidad de los cambios [18].

Una práctica habitual en sistemas que mantienen historial de datos es el **borrado lógico**: en lugar de eliminar físicamente un registro, se activa un campo booleano que lo marca como inactivo. De este modo el registro desaparece de la interfaz y de las consultas activas, pero permanece en la base de datos preservando la integridad referencial de todos los registros históricos que lo referencian [8].

3.7. Integración de servicios mediante APIs externas

Una API (*Application Programming Interface*) actúa como una interfaz de comunicación que permite a una aplicación solicitar datos o servicios a otra. En las APIs web modernas, esta interacción suele realizarse mediante peticiones HTTP y las respuestas se intercambian habitualmente en formato JSON [21].

Por seguridad, estas llamadas deben realizarse siempre desde el *backend*. Así, las credenciales o claves de acceso nunca llegan al navegador del usuario final, manteniéndose protegidas de posibles filtraciones [35].

La teoría de integración de APIs resulta vital en dos frentes muy distintos del proyecto:

- **Bases de datos colaborativas:** Una base de datos colaborativa accesible mediante API es una fuente de información compartida que permite a sistemas externos consultar o utilizar dichos datos de forma estructurada y automatizada, favoreciendo la reutilización de la información y la interoperabilidad entre aplicaciones.
- **Modelos de Lenguaje (LLM):** Un Modelo de Lenguaje de Gran Tamaño (LLM, *Large Language Model*) es un sistema de inteligencia artificial entrenado con grandes cantidades de texto para comprender y generar lenguaje natural, basado hoy en día mayoritariamente en la arquitectura de red neuronal Transformer [43]. Gracias a este entrenamiento, puede interpretar instrucciones, responder preguntas y producir contenido coherente en distintos contextos. Estos modelos pueden utilizarse mediante una API, permitiendo que un servidor envíe solicitudes con información de entrada y reciba respuestas generadas automáticamente sin necesidad de ejecutar el modelo localmente.

El servidor se conecta a la API de un LLM y le envía un *prompt*, obteniendo una respuesta construida a partir de patrones aprendidos durante su entrenamiento [1]. Para integrar de forma fiable estos modelos en una aplicación, resulta fundamental el uso de técnicas como el *structured output prompting* [46], mediante las cuales se especifica que la respuesta debe seguir un formato estructurado, como JSON, en lugar de texto libre. Además, es habitual configurar una **temperatura** baja, es decir, un nivel reducido de aleatoriedad en la generación de respuestas, para reducir la variabilidad y minimizar la aparición de información incorrecta o alucinaciones generadas por el modelo [11].

3.8. Conceptos de nutrición y dietética

Macronutrientes y balance calórico

Los macronutrientes son los componentes principales de los alimentos que el organismo utiliza como fuente de energía y como materia prima para sus funciones vitales. Se distinguen tres grupos [47]:

Proteínas Son necesarias para la construcción y reparación de tejidos (4 kcal/g).

Carbohidratos Constituyen la principal fuente de energía del organismo (4 kcal/g).

Grasas Sirven como reserva energética y contribuyen a diversas funciones hormonales (9 kcal/g).

El **balance calórico** es la relación entre las calorías consumidas a través de la alimentación y las calorías gastadas por el organismo. Cuando la ingesta supera al gasto se produce un superávit calórico, que el organismo almacena principalmente en forma de grasa. Cuando el gasto supera a la ingesta se produce un déficit calórico, que obliga al organismo a recurrir a sus reservas energéticas. El control del balance calórico y de la distribución de macronutrientes es la base del seguimiento nutricional preciso [14].

Cálculo del gasto calórico: la fórmula de Mifflin–St Jeor

El metabolismo basal (MB) es la cantidad mínima de energía que el organismo necesita para mantener sus funciones vitales en reposo absoluto. Su estimación es el punto de partida para calcular los requerimientos calóricos diarios de una persona [30]. La fórmula de Mifflin–St Jeor, publicada en 1990, destaca por su precisión en población general y es la más empleada en el ámbito clínico y nutricional [30]:

- Hombres: $MB = 10 \times \text{peso (kg)} + 6,25 \times \text{altura (cm)} - 5 \times \text{edad (años)} + 5$
- Mujeres: $MB = 10 \times \text{peso (kg)} + 6,25 \times \text{altura (cm)} - 5 \times \text{edad (años)} - 161$

El **gasto calórico total** (GET) se obtiene multiplicando el metabolismo basal por un **factor de actividad** que refleja el nivel de ejercicio físico habitual de la persona. Este factor oscila típicamente entre 1,2 para personas sedentarias y 1,9 para personas con actividad física muy intensa. El GET representa las calorías diarias necesarias para mantener el peso actual; es el valor de referencia a partir del cual se aplican los ajustes correspondientes a cada objetivo nutricional.

3.9. Objetivos nutricionales y patrones dietéticos

El objetivo nutricional determina la relación entre la ingesta calórica y el gasto energético del usuario [14]. Dependiendo de las necesidades individuales, puede orientarse a la pérdida de grasa mediante un déficit calórico, a la ganancia muscular mediante un superávit energético, al mantenimiento del peso corporal mediante un equilibrio entre consumo y gasto, o a metas más específicas como la recomposición corporal y la mejora del rendimiento deportivo. También es posible seguir una alimentación sin restricciones concretas, centrada en mantener hábitos saludables y equilibrados.

Por otro lado, el patrón dietético establece qué tipos de alimentos se priorizan o se limitan dentro de la alimentación diaria, independientemente del objetivo nutricional [22]. Estos patrones pueden basarse en el origen de los alimentos, como ocurre en las dietas vegetariana, vegana o pescetariana; en la restricción de determinados macronutrientes, como en las dietas cetogénica o paleolítica; o en la búsqueda de beneficios para la salud, destacando ejemplos como la dieta mediterránea, la cardiosaludable, la antiinflamatoria o la baja en FODMAPs [5]. De este modo, un mismo objetivo nutricional puede combinarse con distintos patrones dietéticos según las preferencias y necesidades de cada persona.

Capítulo 4

Técnicas y herramientas

En este capítulo se detallan las herramientas tecnológicas y los lenguajes seleccionados para el desarrollo de SoulFoodAI. Se explicará por qué se han escogido estas tecnologías en concreto y qué papel fundamental desempeñan en la arquitectura del proyecto.

4.1. Tecnologías de *Backend* y Persistencia

ASP.NET Core 9.0 (C#)

Para el desarrollo de la API REST que constituye el núcleo de la aplicación se seleccionó **C#** [25] junto con **ASP.NET Core 9.0** [23]. Esta tecnología ofrece un sistema de tipado fuerte y un alto rendimiento, aspectos especialmente relevantes para implementar de forma segura la lógica de negocio y los cálculos relacionados con la generación de planes nutricionales. Además, el sistema de inyección de dependencias incorporado en ASP.NET Core permitió estructurar la aplicación de manera modular.

Entity Framework Core y Microsoft SQL Server

La información almacenada en SoulFoodAI sigue un modelo relacional, ya que existen dependencias entre distintas entidades del sistema, como usuarios, planes nutricionales, días, recetas e ingredientes. Debido a esta estructura, se seleccionó **Microsoft SQL Server** [27] como sistema gestor de bases de datos, proporcionando un entorno fiable para almacenar y relacionar la información de forma consistente.

El acceso a los datos se implementó mediante **Entity Framework Core** [26], un ORM que permite trabajar con la base de datos utilizando objetos y clases de C# en lugar de consultas SQL manuales. Para ello, se adoptó el enfoque *Code First*, definiendo las entidades y sus relaciones directamente en el código y delegando en el ORM la creación y actualización del esquema de la base de datos. Asimismo, Entity Framework Core facilitó la implementación del borrado lógico (*soft delete*), permitiendo conservar la información histórica del sistema sin eliminar físicamente los registros asociados.

4.2. Tecnologías de *Frontend* e Interfaz

Angular y Angular CDK

El cliente web se desarrolló siguiendo el modelo de *Single Page Application* (SPA) para permitir la actualización de la interfaz en tiempo real sin necesidad de recargar la página. Para ello se seleccionó **Angular** [13], un *framework* basado en TypeScript que proporciona una arquitectura modular orientada a componentes.

El desarrollo de interacciones avanzadas, como el sistema de arrastrar y soltar (*drag and drop*) del calendario nutricional, se apoyó en **Angular CDK** [12] (*Component Dev Kit*). Esta librería proporciona comportamientos interactivos preconstruidos, facilitando la gestión de eventos del ratón y dispositivos táctiles de forma nativa dentro del ciclo de vida de Angular.

CSS Nativo y Metodología BEM

La maquetación de la interfaz gráfica se implementó utilizando exclusivamente **CSS nativo**, descartando el uso de librerías de estilos predefinidos como Bootstrap o Tailwind CSS. Esta decisión permite mantener un control estricto sobre el diseño visual y reduce la dependencia de paquetes externos. Para organizar el código CSS y evitar colisiones de estilos entre los distintos componentes, se adoptó la metodología de nombrado **BEM** [48] (*Block, Element, Modifier*).

4.3. Inteligencia Artificial y Servicios Externos

API de Google Gemini 2.5 Flash

La inteligencia de la aplicación se centralizó mediante la integración del *backend* con la API del modelo de lenguaje **Gemini 2.5 Flash** [11] de Google. El modelo interviene en tres flujos dinámicos específicos: la creación de recetas adaptadas a la despensa, el cuadro automatizado de macronutrientes modificando cantidades, y la adaptación de los objetivos nutricionales en base al reporte semanal del usuario. Durante la fase de análisis se evaluaron distintas APIs de modelos de lenguaje atendiendo principalmente a criterios de coste, disponibilidad para pruebas y capacidad de integración. A continuación, se presenta una comparativa de algunos de los modelos más representativos disponibles durante el desarrollo del proyecto.

Modelo	Entrada (\$/1M)	Salida (\$/1M)	Contexto	Capa Gratuita	Enfoque principal
GPT-4.1	2,00	8,00	128K	No	Razonamiento complejo
GPT-4.1 Mini	0,40	1,60	128K	No	Tareas rápidas y económicas
Claude Sonnet 4.6	3,00	15,00	200K	No	Alta precisión analítica
Gemini 2.5 Pro	1,25	5,00	2M	Limitada	Procesamiento masivo de datos
Gemini 2.5 Flash	0,30	2,50	1M	Sí	Equilibrio óptimo coste/velocidad

Tabla 4.1: Comparativa técnica y económica de APIs de modelos de lenguaje.

Tras analizar las distintas alternativas, se optó por utilizar **Gemini 2.5** debido a que ofrecía una de las cuotas gratuitas más amplias para desarrollo y pruebas, además de unos costes significativamente inferiores a los de otras soluciones equivalentes. Durante la implementación se comprobó que su rendimiento era suficiente para realizar todas las funcionalidades de la aplicación, por lo que proporcionaba la mejor relación entre coste, disponibilidad y prestaciones para las necesidades de SoulFoodAI.

API de Open Food Facts

Para nutrir el catálogo de ingredientes sin necesidad de mantener una base de datos estática propia, la aplicación se conecta a la API pública de **Open Food Facts** [34], una plataforma colaborativa que alberga la información nutricional de más de 4 millones de productos comerciales. Esta integración permite a los usuarios realizar búsquedas mediante cadenas de

texto que combinan nombres de alimentos, marcas o supermercados. Al procesar la consulta, el sistema recupera al instante los valores exactos de calorías y macronutrientes del producto seleccionado, permitiendo incorporarlo directamente a la despensa virtual personalizada.

4.4. Gestión del Proyecto, Pruebas y Calidad

Zube y Metodología Scrum

La gestión del desarrollo se basó en la metodología ágil **Scrum** [36], organizando el trabajo en iteraciones o *sprints*. Para el seguimiento de las tareas se empleó **Zube** [49], una plataforma de gestión de proyectos integrada con GitHub. A través de su tablero Kanban (véase la Figura 4.1), los requisitos y errores (*issues*) del sistema se representaron como tarjetas accionables, facilitando la priorización y trazabilidad del trabajo en cada iteración.

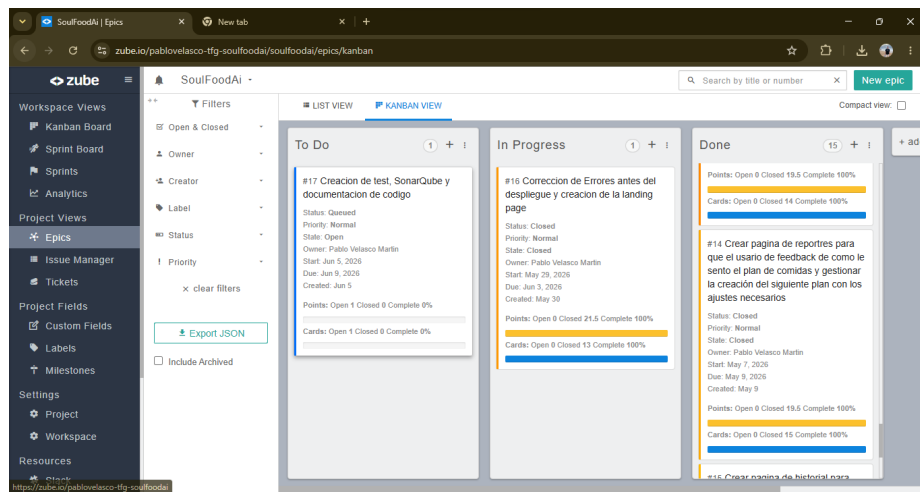


Figura 4.1: Tablero Kanban utilizado para la gestión y seguimiento iterativo del proyecto en Zube. Elaboración propia.

Entornos, Versionado y Documentación de API

El entorno de desarrollo se dividió según la capa arquitectónica: **Visual Studio 2022** [29] para la API en C# y **Visual Studio Code** [28] para

el cliente en Angular. El control de versiones del código fuente se gestionó mediante **Git** [38], utilizando **GitHub** [10] como repositorio remoto.

La documentación y prueba de la API REST se realizó integrando **Swagger** [37] en ASP.NET Core. Esta herramienta genera automáticamente una interfaz web interactiva (véase la Figura 4.2) que permite consultar los *endpoints* disponibles y validar el uso de *tokens* JWT.

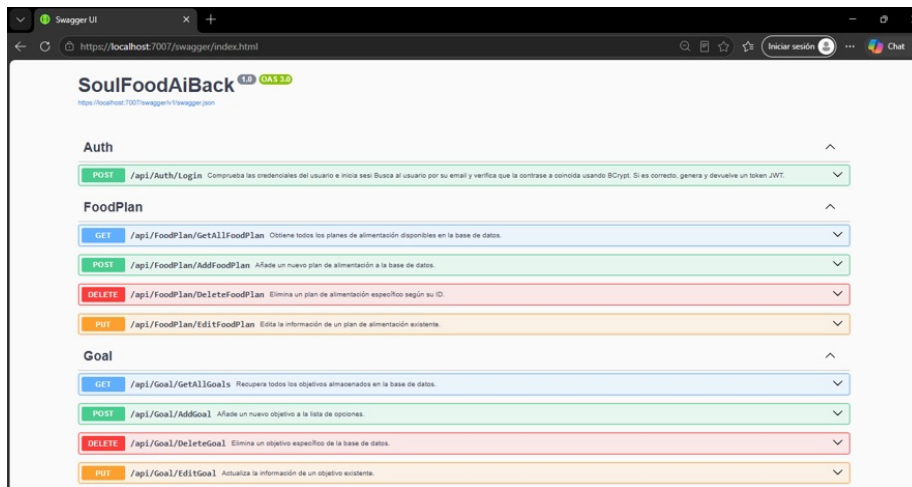


Figura 4.2: Interfaz Swagger utilizada para documentar y probar los *endpoints* de la API REST. Elaboración propia.

Finalmente, la redacción de la presente memoria técnica se realizó mediante el sistema de composición **LaTeX** [17] utilizando el editor colaborativo **Overleaf**.

Marcos de Pruebas Automáticas (xUnit y Vitest)

El aseguramiento del correcto funcionamiento del sistema se llevó a cabo mediante pruebas unitarias en ambas capas de la aplicación. En el *backend*, se utilizó **xUnit** [33] (véase la Figura 4.3) para verificar la lógica de negocio, como el cálculo de calorías y macronutrientes, de forma aislada a la base de datos. En el *frontend*, se implementó **Vitest** [45], un entorno de pruebas unitarias diseñado para herramientas de compilación modernas, permitiendo la validación ágil de los componentes de Angular.

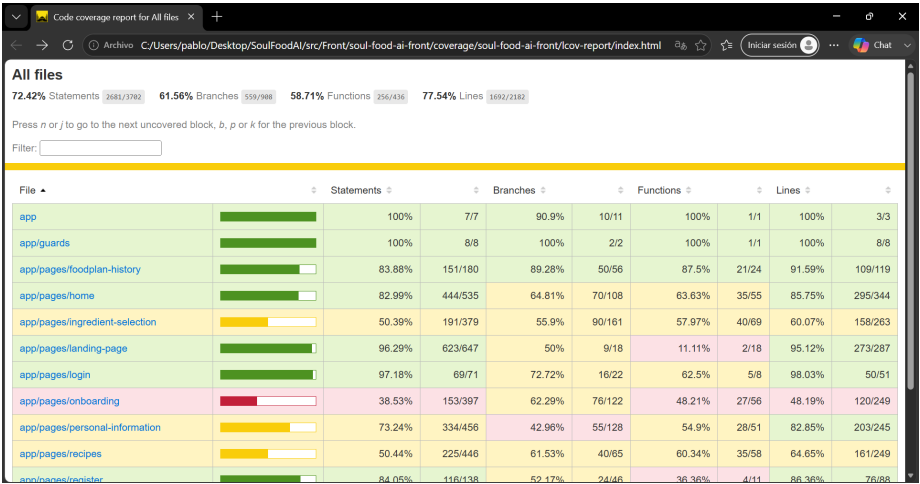


Figura 4.3: Ejecución de las baterías de pruebas unitarias del *backend* mediante xUnit. Elaboración propia.

Análisis Estático (SonarCloud)

El análisis estático del código se delegó a **SonarCloud** [40] (véase la Figura 4.4), un servicio en la nube encargado de la inspección continua del repositorio. Esta herramienta automatiza la detección de vulnerabilidades, *code smells* y deuda técnica, al mismo tiempo que centraliza las métricas de cobertura de pruebas (*Code Coverage*) generadas por xUnit y Vitest.

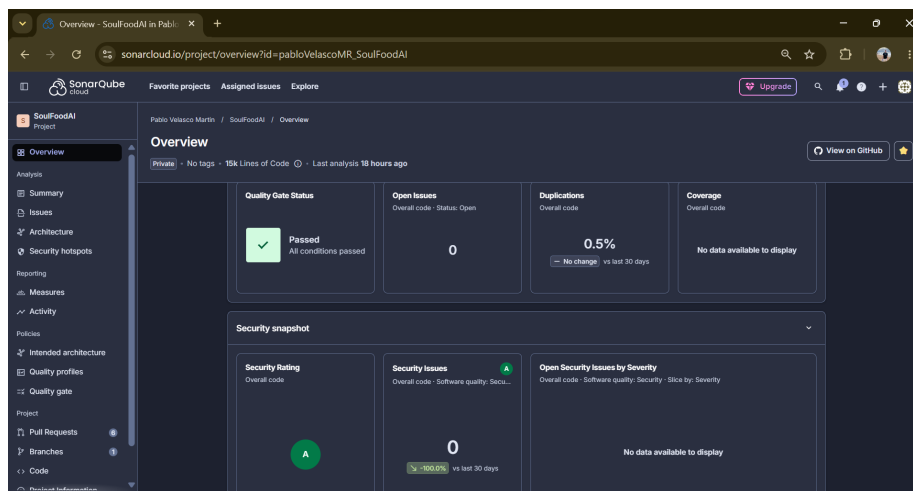


Figura 4.4: Panel de control de SonarCloud mostrando el estado de calidad del código, deuda técnica y cobertura de pruebas del proyecto. Elaboración propia.

4.5. Infraestructura y Despliegue

Contenedorización con Docker

Para estandarizar los entornos de ejecución, se utilizó **Docker** [2]. La contenedorización permite empaquetar la aplicación junto con sus dependencias en unidades aisladas, garantizando un comportamiento consistente entre el desarrollo local y el servidor de producción. En este proyecto, Docker se empleó para virtualizar la base de datos SQL Server y el *backend*, facilitando su configuración y despliegue.

Alojamiento en la Nube (Azure y Vercel)

El despliegue de la aplicación se realizó separando físicamente el cliente y el servidor en plataformas especializadas:

Microsoft Azure (*backend*)

La API REST se desplegó mediante *Azure App Service* [24], y la base de datos se alojó en *Azure SQL Database*, aprovechando la compatibilidad nativa de estos servicios con el ecosistema .NET y SQL Server (véase la Figura 4.5).

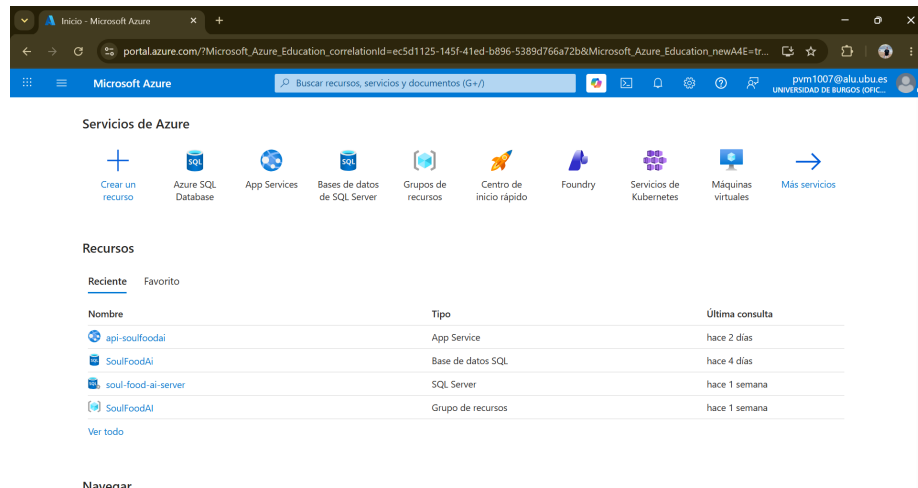


Figura 4.5: Panel de recursos y servicios del *backend* alojados en Microsoft Azure. Elaboración propia.

Vercel (*frontend*)

La aplicación de Angular se alojó en **Vercel** [44], una plataforma optimizada para sitios estáticos y *frameworks frontend*. Su integración con GitHub permite el despliegue automático del código cliente cada vez que se actualiza el repositorio principal.

Capítulo 5

Aspectos relevantes del desarrollo del proyecto

En este capítulo se exponen las decisiones técnicas más relevantes tomadas durante el desarrollo: cómo evolucionó el modelo de datos, por qué se tomaron determinadas decisiones arquitectónicas y de qué forma se integró el motor de inteligencia artificial, entre otras.

5.1. Inicio del proyecto y decisiones de diseño

Como se expuso en la introducción, en la actualidad, la inmensa mayoría de las aplicaciones comerciales actúan como simples calculadoras matemáticas, imponiendo dietas rígidas enfocadas de forma exclusiva en la pérdida de peso y proponiendo recetas estáticas que exigen comprar ingredientes poco comunes que acaban pudriéndose en el fondo de la nevera. Esta desconexión con la realidad de la despensa, sumada a la total falta de adaptabilidad ante el estrés, el nivel de energía o la calidad del sueño del usuario, genera una profunda frustración, especialmente en personas jóvenes o recién independizadas que simplemente buscan aprender a comer de forma saludable y variada sin caer en la monotonía.

Desde el punto de vista técnico, comprender la magnitud de este problema definió el enfoque principal durante las primeras reuniones con la tutora: la plataforma no podía limitarse a ofrecer un registro convencional de alimentos. Su verdadero valor diferencial, y su mayor reto arquitectónico, debía ser la construcción de un sistema dinámico capaz de generar recetas a partir de

los ingredientes reales del usuario, evaluar su estado físico y emocional cada semana mediante un reporte, y reajustar automáticamente su planificación nutricional. Eso fijó desde el principio dos requisitos no negociables: una arquitectura asíncrona capaz de gestionar llamadas a APIs externas y un modelo de datos lo suficientemente flexible para registrar y reajustar la evolución del usuario semana a semana.

En cuanto a la arquitectura del servidor, dado que el desarrollo recaía sobre una única persona, se optó por *fat controllers*: la lógica de negocio y el acceso a datos residen directamente en los controladores, evitando capas intermedias que habrían añadido complejidad sin aportar valor real a la escala del proyecto. Esta elección resultó ser un gran acierto para el proyecto, ya que evitó la sobreingeniería típica de crear múltiples capas intermedias que habrían añadido una complejidad innecesaria a esta escala.

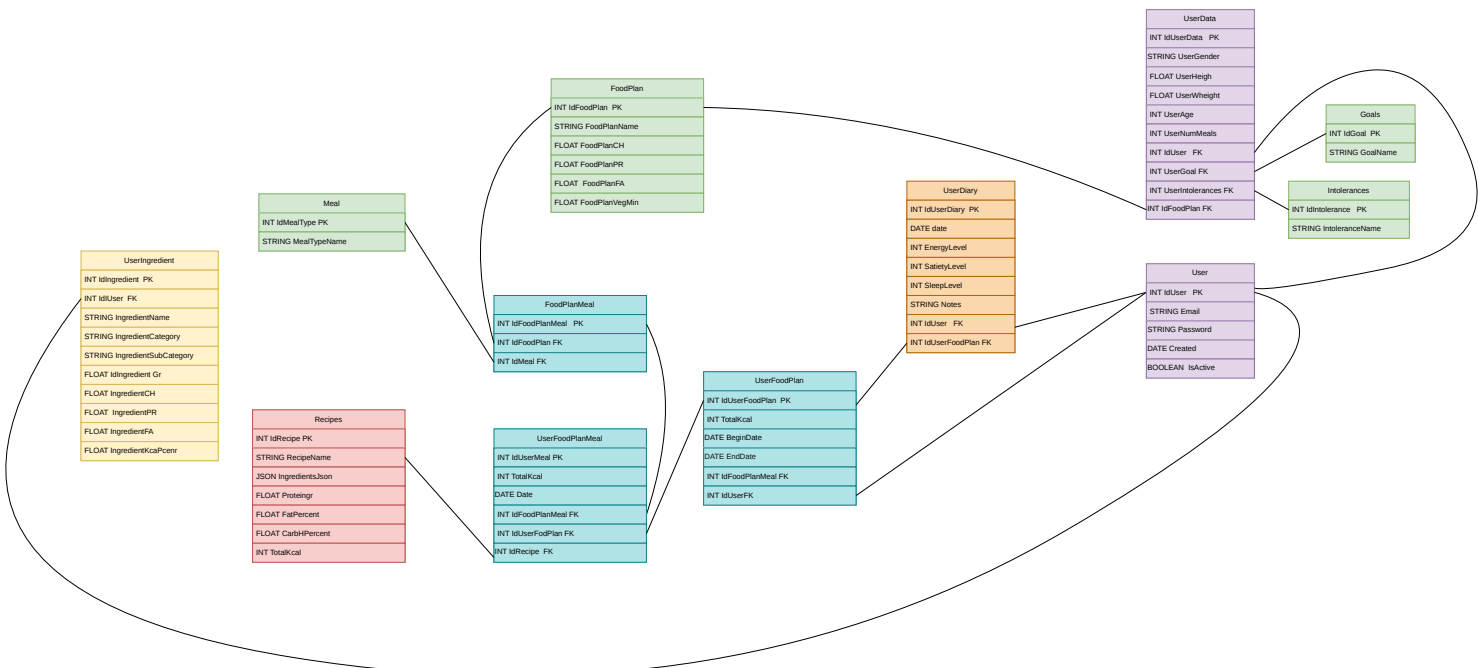
5.2. Evolución del modelo de datos

El diseño de la base de datos no fue un proceso lineal. Para su desarrollo se adoptó el enfoque *Code First* mediante Entity Framework Core, por lo que el esquema de la base de datos partió de un primer modelado y la creación de esos modelos como clases en código. Esta metodología facilitó que la estructura fuese evolucionando de forma ágil a medida que la implementación de nuevas funcionalidades revelaba limitaciones en el diseño inicial.

Estructura y relaciones del modelo inicial

El primer modelo de datos presentaba graves limitaciones prácticas: al intentar implementar las funcionalidades dinámicas deseadas por el proyecto, se comprobó que la estructura rígida de las tablas impedía escalar el sistema de forma correcta. Para facilitar la comprensión de este punto de partida, se incluye a continuación el diagrama entidad-relación original.

5.2. EVOLUCIÓN DEL MODELO DE DATOS



Las entidades del modelo original se agrupaban en cuatro bloques lógicos, cada uno orientado a una parte concreta del proceso nutricional:

- **Tablas de catálogo base (datos de referencia):** Almacenaban las opciones disponibles durante el *onboarding*: Goals (objetivos físicos), Intolerances (restricciones alimentarias), Meals (tipos de ingesta) y FoodPlans (patrones de dieta predefinidos). Actuaban como diccionarios estáticos consultados por el resto del sistema.
- **Tablas de usuarios y datos personales:** La entidad Users gestionaba la autenticación, mientras que UserData almacenaba los datos del *onboarding*: edad, peso, altura, nivel de actividad y parámetros de cálculo nutricional. Ambas mantenían una relación uno a uno que garantizaba un único perfil por cuenta.
- **Tablas de ingredientes y recetas:** Era la parte más rígida del modelo. No existía un catálogo centralizado de alimentos; cada usuario almacenaba los suyos de forma independiente en UserIngredients. Las recetas guardaban sus ingredientes y cantidades en un campo de texto JSON (IngredientsJson), lo que simplificaba la creación inicial pero bloqueaba las consultas relacionales y los cálculos nutricionales avanzados.
- **Tablas de planificación y seguimiento:** UserFoodPlans era el contenedor principal de la semana. UserFoodPlanMeals representaba cada comida dentro del plan, y UserDiaries recogía el *feedback* del usuario sobre adherencia, saciedad y energía.

Este esquema cubría las funcionalidades previstas en las primeras fases, pero conforme se incorporaron la integración con APIs externas, la reutilización de ingredientes y la generación dinámica de planes, aparecieron limitaciones que exigieron una remodelación profunda.

Limitaciones del modelo inicial

Las restricciones se concentraban en dos frentes.

El primero era la **gestión de ingredientes**. Al no existir un catálogo centralizado, si varios usuarios seleccionaban el mismo alimento sus datos nutricionales se duplicaban tantas veces como usuarios lo eligieran, encareciendo el mantenimiento. La integración con Open Food Facts agravaba el problema: el modelo obligaba a descargar y almacenar cada producto de

forma independiente por usuario, sin reutilizar datos ya obtenidos. Tampoco contemplaba la convivencia entre ingredientes externos e ingredientes creados manualmente, y los borrados físicos destruían referencias históricas en recetas y planes anteriores.

El segundo era la **rigidez de la planificación semanal**. UserFoodPlans funcionaba como un contenedor cerrado donde las comidas quedaban atadas a una semana concreta sin posibilidad de reutilizarse, lo que era incompatible con el objetivo de adaptar continuamente la dieta al usuario. El campo IngredientsJson bloqueaba además la interacción con la IA, que necesitaba operar sobre una representación relacional y flexible, no sobre bloques de texto serializado.

El modelo final

El núcleo de identidad se mantuvo: Users gestiona la autenticación y UserData conserva el perfil nutricional con una relación uno a uno entre ambas.

El cambio principal fue la creación de la entidad centralizada Ingredients, que actúa como repositorio único de alimentos. Cada ingrediente existe una sola vez en el sistema y puede ser referenciado por múltiples usuarios y recetas. UserIngredients pasó a ser exclusivamente una tabla de asociación entre un usuario y los ingredientes de su despensa, sin almacenar datos nutricionales propios.

Las recetas se rediseñaron eliminando el campo JSON y estableciendo relaciones explícitas con los ingredientes, lo que permite consultar, modificar y reutilizar cada componente de forma independiente y recalcular los macros con precisión.

La planificación semanal adoptó un modelo modular: las recetas son elementos independientes que pueden asignarse libremente a distintos días y comidas del calendario sin quedar atadas a una semana concreta. Finalmente se añadieron las entidades de seguimiento, que almacenan los reportes semanales del usuario y alimentan al sistema de IA para proponer ajustes en las siguientes planificaciones.

La Figura 5.2 muestra el resultado final de esta evolución.

5.3. Integración con fuentes externas de información nutricional

Con el modelo de datos normalizado, el siguiente reto fue dotar al sistema de una fuente fiable de información nutricional. Para ello se integró Open Food Facts [34], una base de datos colaborativa y abierta de productos alimentarios accesible mediante API REST.

El flujo de integración parte del *frontend*: el usuario introduce un término de búsqueda, el *backend* construye la consulta hacia la API externa y transforma la respuesta al modelo interno de ingredientes. La Figura 5.3 muestra este proceso completo.

Durante las primeras pruebas se detectó un comportamiento irregular de la API: búsquedas válidas devolvían listas vacías y, segundos después con los mismos parámetros, respondían correctamente. Para resolverlo se implementó un mecanismo de reintentos automáticos con pausa entre intentos, lo que redujo de forma notable las búsquedas fallidas percibidas por el usuario.

Open Food Facts devuelve un volumen elevado de campos por producto. Una capa de adaptación extrae únicamente los relevantes: calorías, proteínas, hidratos de carbono, grasas, marca e imagen. Todos los valores se normalizan a 100 gramos, ya que es la representación con mayor disponibilidad y fiabilidad en el catálogo.

En cuanto a la persistencia, cuando el usuario añade un producto a su despensa el sistema comprueba si ese ingrediente ya existe en la tabla Ingredients. Si es así, crea directamente la asociación en UserIngredients; si no, inserta primero el nuevo registro y después establece el vínculo. Este mecanismo evita duplicaciones y permite que múltiples usuarios compartan la misma entrada nutricional sin almacenarla repetidamente, aprovechando directamente la remodelación descrita en la sección anterior.

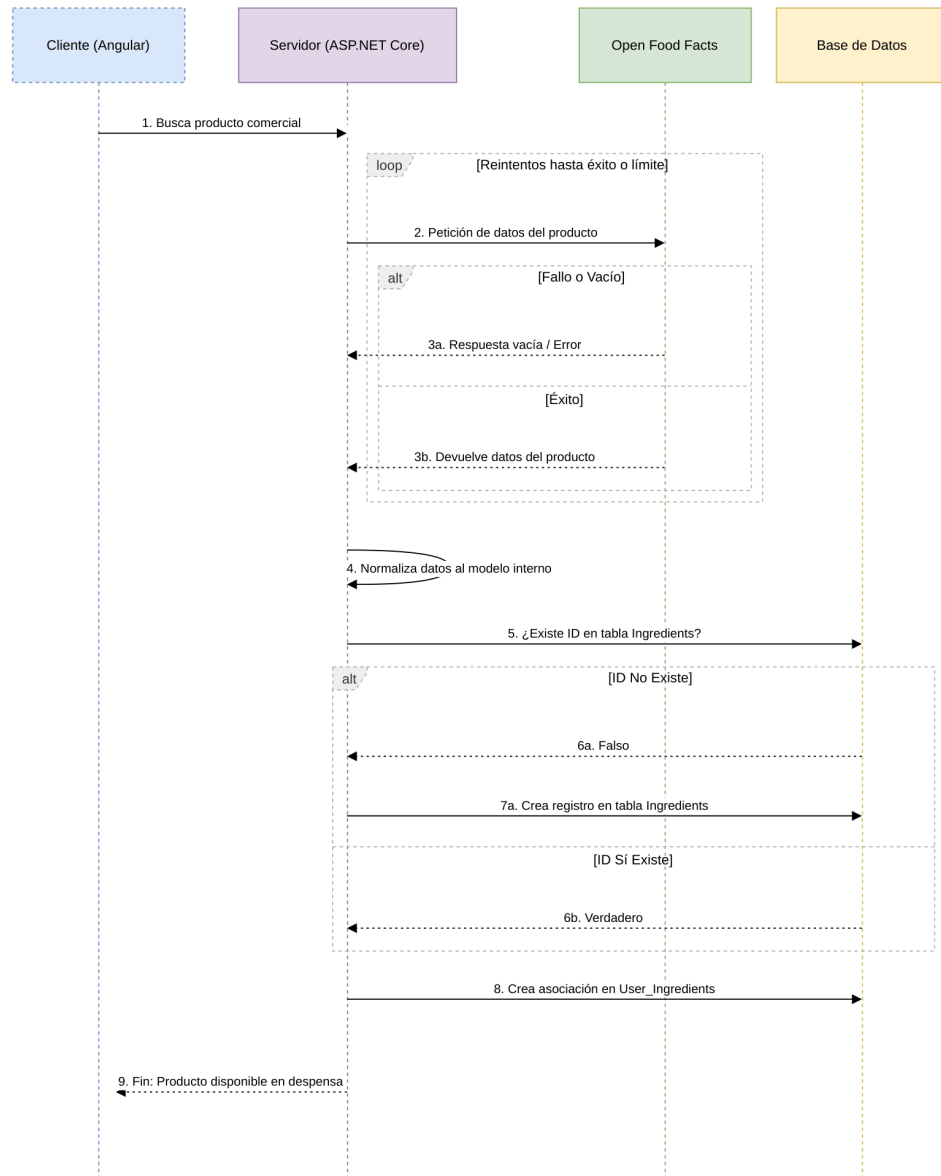


Figura 5.3: Flujo de búsqueda, transformación e incorporación de ingredientes mediante Open Food Facts.

5.4. Integración del motor de inteligencia artificial

La integración de Gemini 2.5 Flash [11] es el elemento técnicamente más complejo del proyecto. El *backend* actúa en todo momento como intermediario: recopila los datos del usuario desde la base de datos, construye el contexto de cada petición y valida las respuestas antes de persistirlas. La IA nunca recibe datos sin filtrar ni interactúa directamente con el cliente. Todas las llamadas se configuraron para devolver JSON estructurado, eliminando la necesidad de procesar lenguaje natural libre. La integración se articula en tres *endpoints* especializados.

Generación de recetas a medida (CreateRecipeAI)

Este módulo genera propuestas culinarias adaptadas al perfil completo del usuario: tipo de comida, objetivo físico, patrón de dieta, intolerancias e ingredientes de la despensa.

Antes de llamar a la API, el *backend* recopila toda esta información y construye un *prompt* estructurado por niveles. El primer nivel define el rol del modelo (chef experto en nutrición y dietética). El segundo aporta el contexto nutricional: tipo de dieta y objetivo físico. El tercero incorpora las intolerancias como restricciones explícitas. El cuarto, y más importante, proporciona la lista exacta de ingredientes de la despensa con instrucción explícita de no utilizar ningún otro. La plantilla base completa de este *prompt* puede consultarse en la Figura 5.5.

Durante las pruebas se observó que los modelos de lenguaje tienden a generar una respuesta incluso cuando las restricciones hacen la petición imposible. Para evitar resultados incoherentes se añadió una regla que obliga a Gemini a devolver un error estructurado en ese caso, de forma que el sistema pueda informar al usuario con claridad.

La gestión de excepciones cubre problemas de conectividad, tiempos de espera, respuestas incompletas y formatos inválidos. Tras recibir la respuesta, el *backend* verifica que todos los identificadores de ingredientes pertenezcan realmente a la despensa del usuario. Los cálculos nutricionales finales los realiza siempre el servidor con los valores almacenados en la base de datos, nunca a partir de los valores que devuelve la IA.

La Figura 5.4 muestra la interacción entre base de datos, *backend* y Gemini durante este proceso.

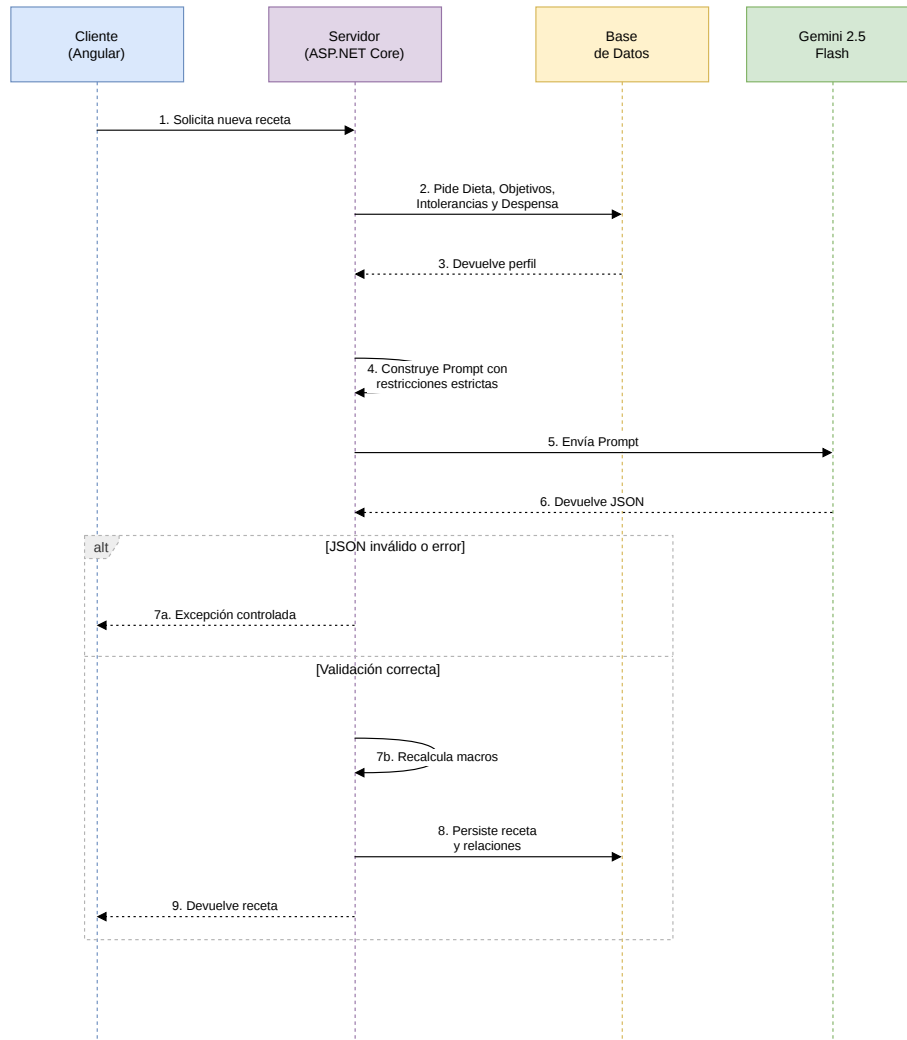


Figura 5.4: Arquitectura general del proceso de generación de recetas mediante Gemini 2.5 Flash.

5.4. INTEGRACIÓN DEL MOTOR DE INTELIGENCIA ARTIFICIAL 19

Eres un chef experto en nutrición y dietética, especialista en diseñar recetas combinando ingredientes y ajustando los macronutrientes según diferentes objetivos.

Tu tarea es crear una receta para la comida: {meal.MealName}.
Debe ser estrictamente compatible con la dieta: {userData.FoodPlan.FoodPlanName}.
Y ten en cuenta que el objetivo de esta persona es: {userData.Goal.GoalName}.

{intoleranciasText}

INSTRUCCIONES DE DISEÑO DE RECETA:

- Sé consecuente y lógico. Las recetas pueden ser elaboraciones complejas o platos muy sencillos del día a día, pero siempre deben ser equilibradas y cumplir con la dieta y alergias asignadas.
- {peticionUsuario}

REGLA Estricta de Ingredientes:

SOLO PUEDES USAR INGREDIENTES DE ESTA LISTA EXACTA: {listIngredientesAviables}.
Bajo ninguna circunstancia incluyas un ingrediente que no está en la lista (solo se puede añadir agua y sal. El aceite si es imprescindible y las especias, si no aparecen ahí, solo se pueden añadir si son opcionales marcándolas con (opcional)).

MANEJO DE CASOS IMPOSIBLES (¡MUY IMPORTANTE!):

Si es físicamente IMPOSIBLE crear una receta que cumpla la dieta Y las intolerancias usando SOLO esos ingredientes (ej: si es celíaco y solo tiene pan de trigo), NO inventes la receta. Devuelve EXACTAMENTE este JSON:

```
{
  "RecipeName": "ERROR",
  "RecipeDescription": "Explica aquí brevemente al usuario por qué...",
  "Ingredients": []
}
```

Si SÍ es posible, devuelve la receta ÚNICAMENTE siguiendo esta estructura JSON exacta. No añadas texto antes ni después, y no uses bloques de código Markdown:

```
{
  "RecipeName": "Nombre creativo y apetecible de la receta",
  "RecipeDescription": "Instrucciones cortas y claras...",
  "Ingredients": [
    {
      "IdIngredient": ID_NUMERICO,
      "QuantityInGrams": CANTIDAD_NUMERICA_ENTERA
    }
  ]
}
```

Figura 5.5: Plantilla base del *prompt* utilizado en el módulo CreateRecipeAI.

Cuadre de macronutrientes diario asistido por IA (AdjustDayMacros)

Cuando el usuario ha colocado sus recetas en el calendario puede ocurrir que la suma de macros de ese día no alcance el objetivo establecido. Este módulo resuelve ese desajuste recalculando automáticamente las cantidades de los ingredientes de las recetas ya seleccionadas, sin crear platos nuevos ni alterar las recetas originales del catálogo. El *backend* calcula la desviación entre los valores objetivo y los aportados por las recetas del día, y construye un *prompt* de carácter matemático que incluye los objetivos nutricionales exactos, los macros de cada receta y el detalle de todos los ingredientes con sus valores por 100 g y las cantidades actuales. Para evitar resultados poco realistas se incorporaron reglas que prohíben modificar especias o ingredientes secundarios y que obligan a mantener raciones dentro de rangos razonables.

Para esta funcionalidad se configuró una temperatura significativamente más baja que en la generación de recetas, priorizando respuestas determinadas frente a la creatividad, ya que el objetivo es optimización matemática y no variedad culinaria. El modelo devuelve una lista JSON con los ingredientes a modificar y sus nuevas cantidades. Antes de aplicar los cambios el *backend* genera una copia ajustada de cada receta afectada, preservando intacta la versión original. Los macronutrientes finales se recalculan siempre desde la base de datos. La Figura 5.6 ilustra este proceso.

5.4. INTEGRACIÓN DEL MOTOR DE INTELIGENCIA ARTIFICIAL

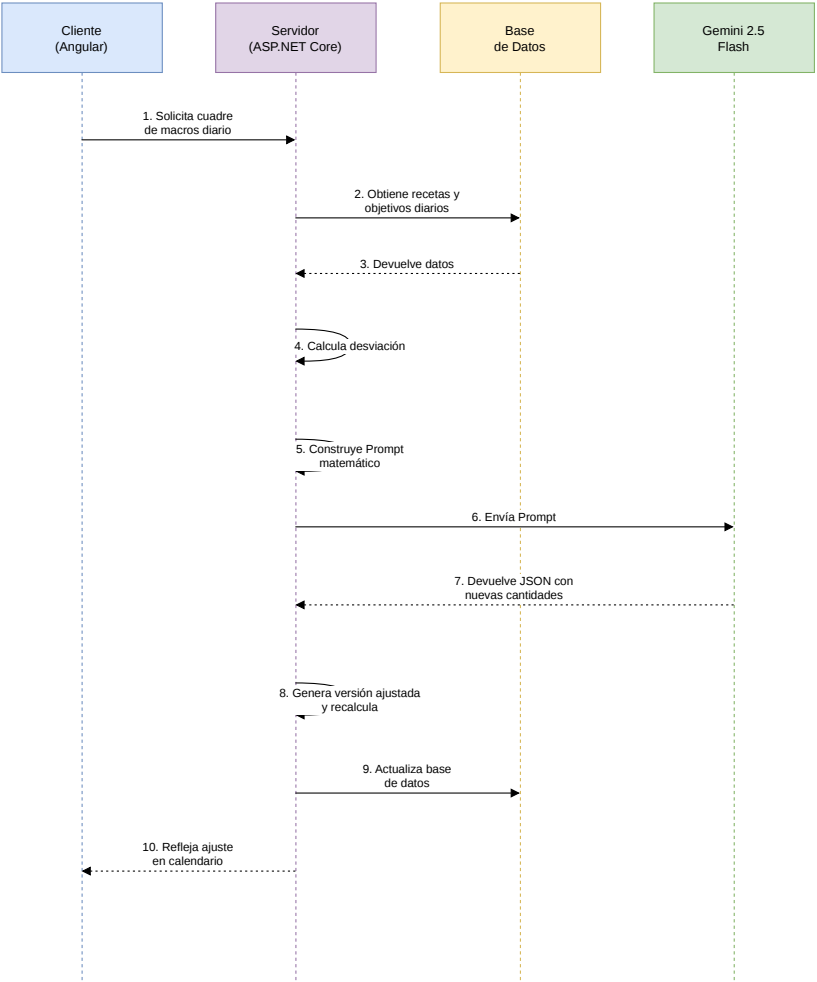


Figura 5.6: Proceso de ajuste automático de cantidades mediante el *endpoint* AdjustDayMacros.

Reporte semanal y ajuste dinámico del plan nutricional (SubmitReportAndCreatePlan)

Este módulo cierra el ciclo semanal. Al terminar la semana el usuario completa un cuestionario con métricas cuantitativas (hambre, energía, calidad del sueño, adherencia y variación de peso) y un campo de texto libre donde describe su experiencia con sus propias palabras. A diferencia de los módulos anteriores, aquí Gemini no trabaja sobre ingredientes ni recetas concretas, sino sobre el comportamiento global del usuario. El *backend* combina los parámetros nutricionales de la semana activa con los datos del cuestionario y construye el *prompt*. El modelo puede así relacionar una adherencia baja con un hambre elevada para deducir que el déficit calórico es excesivo, o asociar bajos niveles de energía con observaciones sobre cansancio para proponer un ajuste en los carbohidratos. Las notas libres reciben un peso especial, ya que capturan matices que las escalas numéricas no pueden recoger.

Gemini devuelve un JSON con dos bloques: la propuesta matemática para la semana siguiente (nuevas calorías diarias y distribución de macros) y un informe en lenguaje natural que justifica cada ajuste. El usuario recibe así no solo un nuevo plan, sino también el razonamiento detrás de cada cambio. Tras validar la respuesta, el *backend* registra el reporte en el historial, almacena el informe de la IA, desactiva la planificación vigente y genera automáticamente la nueva estructura semanal de siete días con los parámetros actualizados. La Figura 5.7 resume este ciclo completo.

5.4. INTEGRACIÓN DEL MOTOR DE INTELIGENCIA ARTIFICIAL43

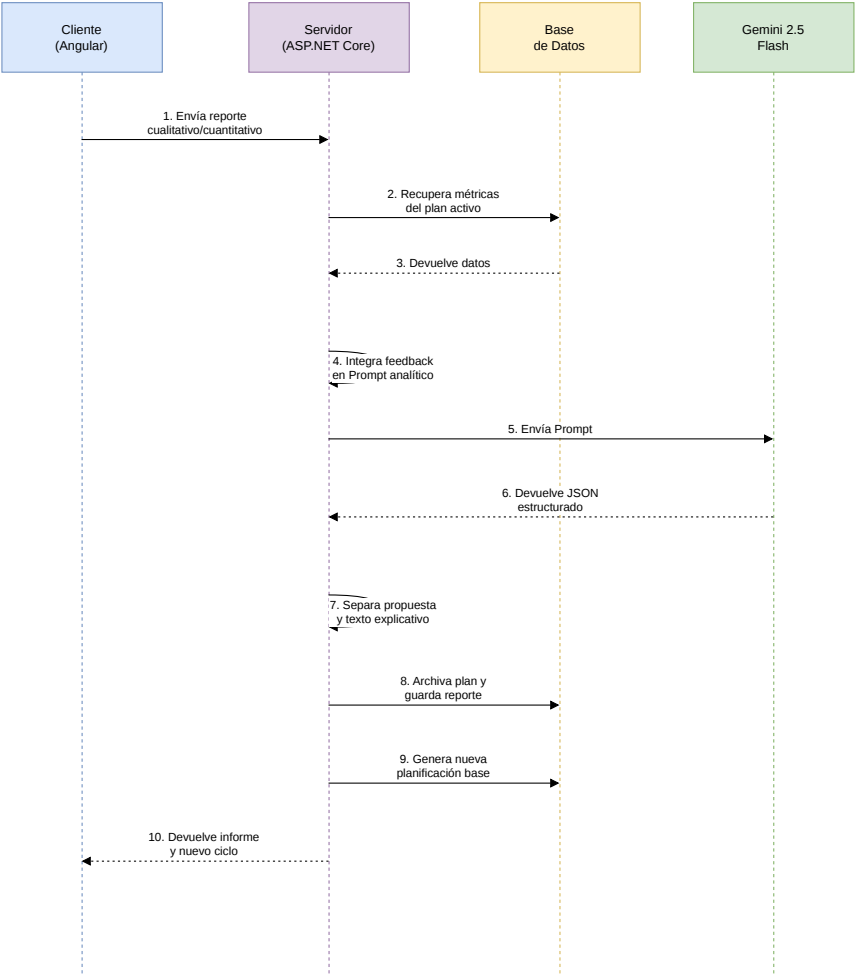


Figura 5.7: Proceso de análisis del reporte semanal y generación automática de la siguiente planificación nutricional.

5.5. Desarrollo de la interfaz de usuario

Arquitectura del frontend

El *frontend* está desarrollado íntegramente en Angular, organizado de forma modular en componentes visuales independientes (panel principal, perfil, catálogo de recetas) y servicios de lógica de negocio en el cliente. La comunicación con el servidor se realiza mediante peticiones HTTP asíncronas en JSON, con un interceptor (`auth.interceptor.ts`) que inyecta automáticamente el token JWT en cada cabecera de autorización. Esta separación permite que la interfaz se centre exclusivamente en capturar interacciones y actualizar vistas, delegando cálculos, integraciones externas y persistencia al *backend*.

Proceso guiado de onboarding

El *onboarding* se presenta como un formulario interactivo por pasos que reduce la carga cognitiva del usuario. El flujo recorre en orden las métricas corporales básicas (edad, género, altura y peso), el estilo de vida (actividad física y número de comidas), el objetivo nutricional, el patrón de dieta y las intolerancias. En cada paso se aplican validaciones reactivas que impiden avanzar con valores incorrectos o físicamente imposibles. Al completar el cuestionario, el bloque de datos se envía al servidor, donde se calculan automáticamente la tasa metabólica basal, las calorías diarias y la distribución de macros.

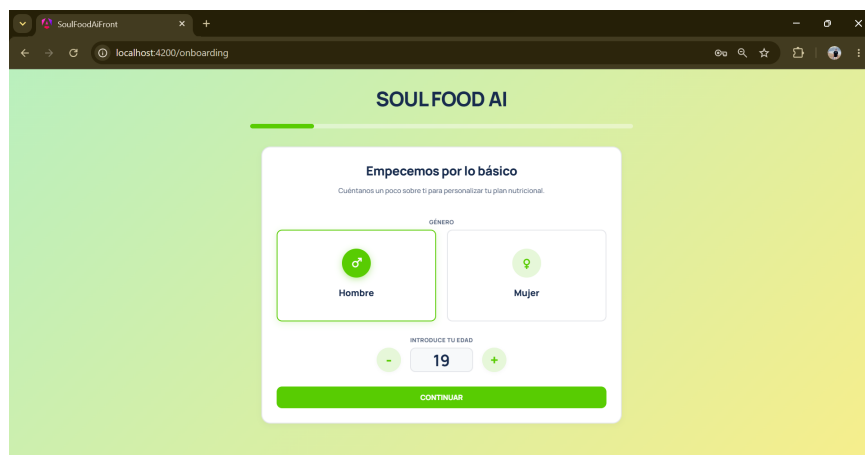


Figura 5.8: Paso de métricas físicas del proceso de *onboarding*.

Gestión de ingredientes y planificación del menú semanal

El usuario construye su despensa seleccionando ingredientes del catálogo base o importando productos comerciales desde Open Food Facts en tiempo real. Sobre esa base crea las recetas que desea consumir.

La planificación del menú tiene lugar en el panel principal mediante un calendario interactivo de siete días con sistema de *drag and drop*: el usuario arrastra las tarjetas de recetas al día deseado y la cabecera dinámica actualiza al instante las calorías y gramos de macros acumulados, mostrando en todo momento la diferencia respecto al objetivo diario.

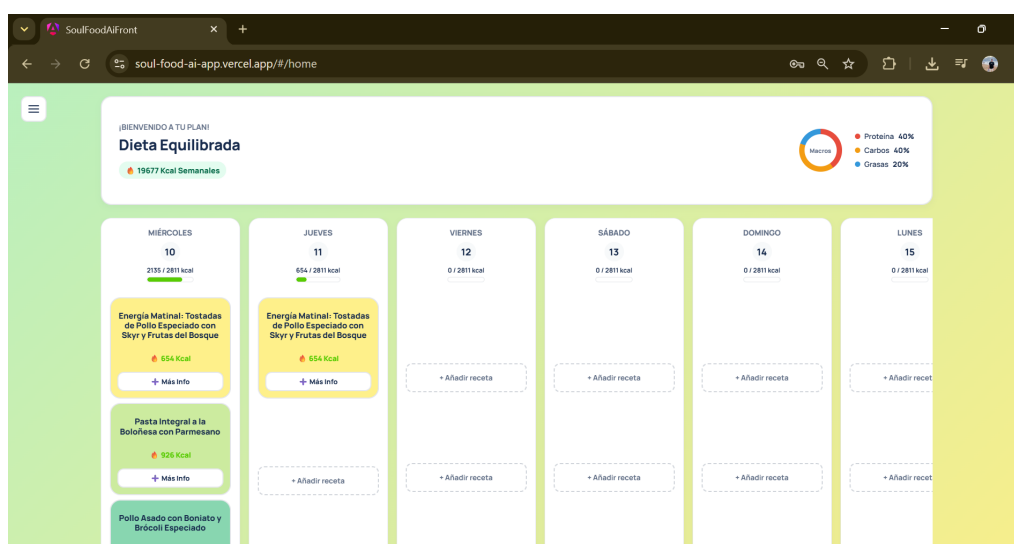


Figura 5.9: Panel principal con el calendario interactivo y las tarjetas de recetas disponibles para arrastrar y soltar.

De este modo, la interfaz de usuario materializa de forma accesible toda la complejidad técnica descrita en las secciones anteriores. Lo que en el servidor implica consultas a la base de datos, integración con repositorios externos y la construcción de *prompts* dinámicos para la inteligencia artificial, en el *frontend* se traduce en una experiencia visual fluida. El usuario simplemente arrastra tarjetas e interactúa con un calendario, mientras el sistema orquesta en segundo plano la adaptación nutricional, cumpliendo así con el objetivo fundacional del proyecto: ofrecer una herramienta inteligente, dinámica y fácil de usar.

Capítulo 6

Trabajos relacionados

En el ámbito de la nutrición digital y la alimentación personalizada pueden identificarse tres grandes categorías de aplicaciones. La primera está formada por herramientas orientadas al seguimiento nutricional y al conteo de calorías. La segunda agrupa plataformas especializadas en la planificación de dietas y menús. Finalmente, en los últimos años ha surgido una tercera categoría basada en inteligencia artificial generativa, capaz de producir recomendaciones y contenido nutricional altamente adaptado al usuario.

Con el objetivo de situar SoulFoodAI dentro de este contexto, a continuación se analizan algunas de las soluciones más representativas del sector y se comparan sus funcionalidades con las implementadas en este proyecto.

6.1. MyFitnessPal

MyFitnessPal [32] es una de las plataformas de seguimiento más utilizadas a nivel global. Destaca por su inmensa base de datos y lectura de códigos de barras, pero su enfoque se limita a la monitorización pasiva. Permite registrar consumos y establecer objetivos, pero la ardua tarea de seleccionar comidas y cuadrar los macronutrientes recae por completo en el usuario, sin ofrecer generación automática ni asistencia inteligente.

6.2. Eat This Much

Eat This Much [3] automatiza la generación de planes alimentarios basándose en calorías objetivo, preferencias y restricciones. Aunque resuelve el problema de la planificación estática, sus menús se generan mediante

algoritmos clásicos de optimización matemática. Por tanto, carece de la adaptabilidad dinámica y el razonamiento contextual que sí ofrecen los modelos de inteligencia artificial modernos.

6.3. MacroFactor

MacroFactor [41] es una aplicación de seguimiento avanzado que ajusta periódicamente las calorías recomendadas analizando la evolución del peso y la ingesta histórica del usuario. Pese a superar la rigidez de las fórmulas metabólicas estáticas, su enfoque sigue siendo puramente cuantitativo y analítico, sin ofrecer generación proactiva de recetas ni soporte de inteligencia artificial.

6.4. Asistentes nutricionales basados en inteligencia artificial

Herramientas conversacionales como ChatGPT o Gemini [1] pueden generar recetas hiperpersonalizadas y planes adaptados a múltiples objetivos mediante instrucciones en lenguaje natural. Sin embargo, su uso aislado presenta una limitación arquitectónica crítica: carecen de memoria estructurada a largo plazo. Al no disponer de una base de datos relacional, no pueden gestionar de forma persistente y determinista la despensa del usuario, su evolución biométrica ni su historial de consumo.

6.5. Comparativa de funcionalidades

La Tabla 6.1 resume las principales características de las aplicaciones analizadas, permitiendo identificar visualmente el ecosistema tecnológico actual.

Tabla 6.1: Comparativa de funcionalidades entre aplicaciones del sector y SoulFoodAI.

Funcionalidad	MyFitnessPal	Eat This Much	MacroFactor	IA Conversacional	SoulFoodAI
Registro nutricional	Sí	Sí	Sí	No	Sí
Cálculo de calorías y macros	Sí	Sí	Sí	Parcial	Sí
Planificación automática de menús	No	Sí	No	Parcial	Sí
Gestión de ingredientes (despensa)	Parcial	Sí	No	No	Sí
Personalización por intolerancias	Sí	Sí	Parcial	Sí	Sí
Generación automática de recetas	No	No	No	Sí	Sí
Integración con APIs alimentarias	No	No	No	No	Sí
Seguimiento semanal	Parcial	Parcial	Sí	No	Sí
Adaptación dinámica del plan	No	Parcial	Sí	No	Sí
Inteligencia artificial generativa	No	No	No	Sí	Sí
Persistencia de datos estructurados	Sí	Sí	Sí	No	Sí

6.6. Posicionamiento y valor añadido de SoulFoodAI

Las aplicaciones analizadas en este capítulo obligan al usuario a adaptar su estilo de vida a las exigencias de la herramienta. Tanto los planificadores estáticos como los contadores de calorías asumen un escenario ideal donde la persona dispone de tiempo y recursos para comprar ingredientes inusuales, sufre un nivel de estrés constante y mantiene una adherencia mecánica a la dieta. Cuando la realidad choca con esta rigidez, el resultado es la frustración y el abandono.

SoulFoodAI invierte este paradigma tecnológico: es el sistema el que se pliega a la realidad del usuario. Al comparar la plataforma con las alternativas del mercado, su valor diferencial no reside únicamente en la tecnología subyacente, sino en cómo esta se orchestra para resolver los problemas prácticos y emocionales del día a día:

1. **Gestión realista de la despensa:** Frente a las aplicaciones que proponen menús cerrados ignorando la disponibilidad de los alimentos, SoulFoodAI permite al usuario construir una despensa con los productos que consume habitualmente. La inteligencia artificial opera bajo una restricción estricta: debe inventar recetas viables y equilibradas utilizando única y exclusivamente lo que ya hay en la nevera. Esto elimina el desperdicio de comida y facilita enormemente la organización, especialmente para perfiles jóvenes o recién independizados.
2. **Acompañamiento integral y dinámico:** Mientras que herramientas como *MyFitnessPal* o *MacroFactor* se limitan a la monitorización

pasiva o al ajuste puramente analítico, SoulFoodAI cierra el ciclo nutricional completo. Acompaña al usuario desde la organización interactiva en el calendario hasta el cuadro automático de raciones diarias, eliminando por completo la carga cognitiva que supone intentar encajar matemáticamente los macronutrientes a mano.

3. **Adaptabilidad empática:** Este es el salto cualitativo más importante respecto a la competencia. En lugar de evaluar el éxito basándose exclusivamente en el peso, el sistema recoge variables subjetivas críticas como la calidad del sueño, los niveles de energía, el hambre o la adherencia. La IA procesa estos matices humanos para recalcular la estrategia de la siguiente semana, promoviendo una relación pacífica y sostenible con la comida en lugar de un régimen punitivo.

En definitiva, SoulFoodAI no busca competir en el saturado mercado de las calculadoras de calorías. Su posicionamiento es el de un **compañero nutricional inteligente** que une la gestión estructurada de datos con la capacidad de razonamiento de Gemini 2.5 Flash [11]. El resultado es una plataforma que hace la alimentación saludable verdaderamente adaptable, económica y, sobre todo, libre de frustraciones.

Capítulo 7

Conclusiones y líneas de trabajo futuras

7.1. Conclusiones

El desarrollo de SoulFoodAI ha permitido construir una plataforma web funcional y desplegada en producción que cubre íntegramente el ciclo nutricional del usuario. La aplicación integra con éxito la gestión estructurada de datos, la conexión con catálogos externos como Open Food Facts y el uso de inteligencia artificial generativa como motor de recomendación.

El pilar más diferenciador del proyecto es su arquitectura de inteligencia artificial. Al posicionar el *backend* como un intermediario estricto entre el usuario y Gemini 2.5 Flash, y mediante la aplicación de *prompt engineering* estructurado, se ha logrado que la IA genere recetas hiperpersonalizadas y ajuste las dietas dinámicamente sin comprometer la integridad matemática de los macronutrientes.

En definitiva, el proyecto ha alcanzado todos los objetivos iniciales, consolidando un prototipo robusto, resiliente y sólidamente preparado para escalar.

7.2. Líneas de trabajo futuras y viabilidad comercial

Aunque el presente proyecto ha sido desarrollado con fines académicos, la arquitectura de la plataforma presenta un alto potencial para su explotación comercial en el emergente sector *FoodTech*.

Se identifican dos grandes vías de mercado para la comercialización de la aplicación. La primera es el modelo **B2C** (usuario particular), enfocado a personas que buscan organizar su alimentación de forma inteligente. La segunda es el modelo **B2B** (profesional), donde dietistas y nutricionistas podrían utilizar SoulFoodAI como herramienta para gestionar a sus pacientes, apoyándose en la IA para pre-generar dietas que posteriormente revisarían y validarían.

Para dar el salto definitivo de prototipo a producto comercial y dar soporte a estos modelos de negocio, las líneas de trabajo futuras deben centrarse en la ampliación estructural del sistema:

- **Implementación de un sistema de roles avanzado:** Desarrollar una jerarquía de usuarios (Administrador, Nutricionista, Paciente, Usuario Gratuito y Usuario Premium) que permita restringir o habilitar el acceso a los distintos módulos de la plataforma según el perfil.
- **Panel de gestión para profesionales (B2B):** Construir un *dashboard* específico para que los nutricionistas puedan enlazar cuentas de pacientes, visualizar su progreso, ajustar manualmente sus macros y revisar las recomendaciones generadas por la IA antes de que el paciente las reciba.
- **Integración de pasarelas de pago:** Implementar un sistema de facturación automatizado (como Stripe o PayPal) para sostener el modelo económico de la aplicación. Esto incluye la gestión de suscripciones mensuales tanto para particulares como para clínicas profesionales.

Bajo esta estructura, el esquema de monetización más adecuado combinaría un modelo *freemium* para particulares con acceso gratuito a la despensa y calendario básico, y suscripción de pago para el uso del motor de IA con un modelo SaaS (*Software as a Service*) por tramos para los profesionales, escalable según su volumen de pacientes.

Bibliografía

- [1] Tom B. Brown et al. Language models are few-shot learners. *Advances in Neural Information Processing Systems*, 33:1877–1901, 2020.
- [2] Docker Inc. Docker documentation [en línea]. Docker. Disponible en: <https://docs.docker.com/>, 2024. Fecha de consulta: mayo de 2025.
- [3] Eat This Much, Inc. Eat this much [en línea]. Eat This Much. Disponible en: <https://www.eatthismuch.com/>, 2024. Fecha de consulta: mayo de 2025.
- [4] ECMA International. ECMA-404: The JSON Data Interchange Syntax. Technical report, ECMA International, 2017.
- [5] Ramón Estruch et al. Primary prevention of cardiovascular disease with a mediterranean diet. *New England Journal of Medicine*, 368(14):1279–1290, 2013.
- [6] Roy Fielding, Jim Gettys, Jeffrey Mogul, Henrik Frystyk, Larry Masinter, Paul Leach, and Tim Berners-Lee. RFC 2616: Hypertext Transfer Protocol– HTTP/1.1. RFC 2616, Internet Engineering Task Force, 1999.
- [7] Roy Thomas Fielding. *Architectural Styles and the Design of Network-based Software Architectures*. PhD thesis, University of California, Irvine, 2000.
- [8] Martin Fowler. *Patterns of Enterprise Application Architecture*. Addison-Wesley, 2002.

- [9] Erich Gamma, Richard Helm, Ralph Johnson, and John Vlissides. *Design Patterns: Elements of Reusable Object-Oriented Software*. Addison-Wesley, 1994.
- [10] GitHub, Inc. Github [en l nea]. GitHub. Disponible en: <https://github.com/>, 2024. Fecha de consulta: mayo de 2025.
- [11] Google DeepMind. Gemini api documentation [en l nea]. Google AI for Developers. Disponible en: <https://ai.google.dev/gemini-api/docs/models>, 2024. Fecha de consulta: mayo de 2025.
- [12] Google LLC. Angular cdk documentation [en l nea]. Angular Material. Disponible en: <https://material.angular.io/cdk/>, 2024. Fecha de consulta: mayo de 2025.
- [13] Google LLC. Angular documentation [en l nea]. Angular. Disponible en: <https://angular.dev/overview>, 2024. Fecha de consulta: mayo de 2025.
- [14] Eric R. Helms, Alan A. Aragon, and Peter J. Fitschen. Evidence-based recommendations for natural bodybuilding contest preparation: nutrition and supplementation. *Journal of the International Society of Sports Nutrition*, 11(1):20, 2014.
- [15] Michael Jones, John Bradley, and Nat Sakimura. RFC 7519: JSON Web Token (JWT). RFC 7519, Internet Engineering Task Force, 2015.
- [16] Michael B. Jones and Dick Hardt. RFC 6750: The OAuth 2.0 Authorization Framework: Bearer Token Usage. RFC 6750, Internet Engineering Task Force, 2012.
- [17] Leslie Lamport. * TEX: A Document Preparation System*. Addison-Wesley, 2 edition, 1994.
- [18] Julia Lerman and Rowan Miller. *Programming Entity Framework: Code First*. O'Reilly Media, 2013.
- [19] Ethan Marcotte. *Responsive Web Design*. A Book Apart, 2011.
- [20] Robert C. Martin. *Agile Software Development, Principles, Patterns, and Practices*. Prentice Hall, 2003.
- [21] Mark Mass  . *REST API Design Rulebook*. O'Reilly Media, 2011.

- [22] Vesanto Melina, Winston Craig, and Susan Levin. Position of the academy of nutrition and dietetics: Vegetarian diets. *Journal of the Academy of Nutrition and Dietetics*, 116(12):1970–1980, 2016.
- [23] Microsoft Corporation. Asp.net core documentation [en l'Ànea]. Microsoft Learn. Disponible en: <https://learn.microsoft.com/en-us/aspnet/core/introduction-to-aspnet-core>, 2024. Fecha de consulta: mayo de 2025.
- [24] Microsoft Corporation. Azure app service documentation [en l'Ànea]. Microsoft Learn. Disponible en: <https://learn.microsoft.com/en-us/azure/app-service/overview>, 2024. Fecha de consulta: mayo de 2025.
- [25] Microsoft Corporation. C# documentation [en l'Ànea]. Microsoft Learn. Disponible en: <https://learn.microsoft.com/en-us/dotnet/csharp/>, 2024. Fecha de consulta: mayo de 2025.
- [26] Microsoft Corporation. Entity framework core documentation [en l'Ànea]. Microsoft Learn. Disponible en: <https://learn.microsoft.com/en-us/ef/core/>, 2024. Fecha de consulta: mayo de 2025.
- [27] Microsoft Corporation. Sql server documentation [en l'Ànea]. Microsoft Learn. Disponible en: <https://learn.microsoft.com/en-us/sql/sql-server/>, 2024. Fecha de consulta: mayo de 2025.
- [28] Microsoft Corporation. Visual studio code [en l'Ànea]. Visual Studio Code. Disponible en: <https://code.visualstudio.com/>, 2024. Fecha de consulta: mayo de 2025.
- [29] Microsoft Corporation. Visual studio documentation [en l'Ànea]. Microsoft Learn. Disponible en: <https://learn.microsoft.com/en-us/visualstudio/>, 2024. Fecha de consulta: mayo de 2025.
- [30] Mark D. Mifflin, Sachiko T. St Jeor, Lisa A. Hill, Barbara J. Scott, Sandra A. Daugherty, and Young O. Koh. A new predictive equation for resting energy expenditure in healthy individuals. *The American Journal of Clinical Nutrition*, 51(2):241–247, 1990.
- [31] Michael S. Mikowski and Josh C. Powell. *Single Page Web Applications*. Manning Publications, 2013.
- [32] MyFitnessPal, Inc. Myfitnesspal [en l'Ànea]. MyFitnessPal. Disponible en: <https://www.myfitnesspal.com/>, 2024. Fecha de consulta: mayo de 2025.

- [33] .NET Foundation. xunit.net [en l'Ànea]. xUnit. Disponible en: <https://xunit.net/>, 2024. Fecha de consulta: mayo de 2025.
- [34] Open Food Facts Contributors. Open food facts api documentation [en l'Ànea]. Open Food Facts. Disponible en: <https://wiki.openfoodfacts.org/API>, 2012. Fecha de consulta: mayo de 2025.
- [35] OWASP Foundation. Owasp top ten 2021 [en l'Ànea]. OWASP Foundation. Disponible en: <https://owasp.org/www-project-top-ten/>, 2021. Fecha de consulta: mayo de 2025.
- [36] Ken Schwaber and Jeff Sutherland. *The Scrum Guide*. Scrum.org, 2020.
- [37] SmartBear Software. Swagger documentation [en l'Ànea]. Swagger. Disponible en: <https://swagger.io/docs/specification/about/>, 2024. Fecha de consulta: mayo de 2025.
- [38] Software Freedom Conservancy. Git [en l'Ànea]. Git. Disponible en: <https://git-scm.com/>, 2024. Fecha de consulta: mayo de 2025.
- [39] Ian Sommerville. *Software Engineering*. Pearson, 10 edition, 2016.
- [40] SonarSource SA. Sonarcloud [en l'Ànea]. SonarCloud. Disponible en: <https://sonarcloud.io/>, 2024. Fecha de consulta: mayo de 2025.
- [41] Stronger by Science. Macrofactor [en l'Ànea]. MacroFactor. Disponible en: <https://macrofactorapp.com/>, 2024. Fecha de consulta: mayo de 2025.
- [42] Andrew S. Tanenbaum and Maarten Van Steen. *Distributed Systems: Principles and Paradigms*. Prentice Hall, 2 edition, 2007.
- [43] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [44] Vercel Inc. Vercel documentation [en l'Ànea]. Vercel. Disponible en: <https://vercel.com/docs/frameworks/angular>, 2024. Fecha de consulta: mayo de 2025.
- [45] Vitest Contributors. Vitest documentation [en l'Ànea]. Vitest. Disponible en: <https://vitest.dev/>, 2024. Fecha de consulta: mayo de 2025.

- [46] Jules White, Quchen Fu, Sam Hays, Michael Sandborn, Carlos Olea, Henry Gilbert, Ashraf Elnashar, Jesse Spencer-Smith, and Douglas C. Schmidt. A prompt pattern catalog to enhance prompt engineering with chatgpt. *arXiv preprint arXiv:2302.11382*, 2023.
- [47] Eleanor Noss Whitney and Sharon Rady Rolfes. *Understanding Nutrition*. Cengage Learning, 15 edition, 2018.
- [48] Yandex. Bem methodology [en lÃnea]. BEM. Disponible en: <https://en.bem.info/methodology/>, 2024. Fecha de consulta: mayo de 2025.
- [49] Zube. Zube - agile project management for github [en lÃnea]. Zube. Disponible en: <https://zube.io/>, 2024. Fecha de consulta: mayo de 2025.